DIOUANE CYBER SECURITY // DARIJA 26/27 الجهة المنشئة: Diouane Cyber Security Lab

Diouane Cyber Security 2Weeks Bug Bounty

دليل عملي شامل بالدارجة المغربية لاختبار أمن تطبيقات الويب وكشف الثغرات فـ 14 يوم بمنهجية منضبطة ومحددة النطاق

01 كيفاش تخدم بهاد الدليل (المنهجية ديال الخدمة) :

OPERATIONAL METHODOLOGY (DARIJA)

📋 مقدمة الدليل: منهجية الاستخدام والضوابط التشغيلية بالدارجة

هاد الدليل كيقدم ليك مسار عملي متكامل كيبدا من كيفاش تنظم مساحة الخدمة فجهازك، من بعد تقرا ردود الويب (HTTP responses)، وتشغّل الأدوات المناسبة لكل تيكنولوجيا، وتحلل كل نتيجة كتوصل ليها، باش تسالي بتقرير احترافي مبني على أدلة قاطعة وبصمات هاش (SHA-256) قابلة للمراجعة والتدقيق. الدليل كيفترض أن الأدوات مأنستالية عندك مسبقاً، وما فيهش أوامر ديال التثبيت. فيه 55 خطوة تشغيل وتحليل، و9 ديال الأمثلة معطلة خاصة بتحليل المخاطر، ومثال ختامي تطبيقي من 20 خطوة متسلسلة.

العبارة ديال «جميع الأدوات والتقنيات» فالمقدمة كتعني المحتويات اللي كاينين فهاد الدليل، وما كتعنيش حصر لكل أدوات أمن الويب كاملة فالعالم. الإشارة لـ 26/27 هي التسمية ديال هاد النسخة المحدثة (مراجعة تقنية بأكتوبر 2026 وما كتدعيش علم الغيب لسنة 2027). الخيارات والباراميطرات يقدرو يتبدلو شويا على حساب النسخة ديال الأداة اللي عندك.

الخطورة الحرجة (Critical Severity) كتكون مرتبطة بأثر أمني مثبت على السيستيم وشروط استغلال وسياق تطبيق محدد، ماشي بعدد الأدوات اللي طلقتي ولا اسم القالب ولا درجة التنبيه ديال السكانير. هاد الأدوات كيعاونوك غير باش تبني فرضيات وتتأكد منها، وما كيضمنوش بوحدهم تلقى ثغرة حرجة.

النطاقات www.example.comg example.com هما نطاقات توثيقية دارتهم منظمة IANA للشرح، وماشي مختبر محلول باش تتيستي فيه الثغرات. الأوامر اللي مشروحة عليهم كتبين غير السينطاكس والترتيب الصحيح. قبل ما دير أي اتصال حقيقي، خاصك تخدم على هدف مرخص ومصرح به ضمن سياسة البرنامج (Scope) أو فمختبر كتمتلكو. النتائج اللي مشروحة هنا هي نتائج متوقعة ونظرية.

مرجع IANA: الغرض من نطاقات الأمثلة والشرح.

قيد عندك السيرفورات والمسارات المصرح بها والاستثناءات والكونطات المعتمدة قبل ما تبدا. ديما السياسة ديال البرنامج (Policy) كتبقى هي المرجع الأول لأي تيست أو فحص للصلاحيات. إلا كان الدومين داخل فالسكوب، هادشي ما كيعطيكش الضوء الأخضر باش تغرق السيت بطلبات لا نهائية ولا تخمن الكودات من قواميس ضخمة ولا تغير الداتا ديال الإنتاج.

مرجع HackerOne: قواعد السلوك والتعامل مع السكوب والأثر الأمني.

▪ الخدمة أمر بأمر وبشويا عليك

الأوامر كتكتب داخل Bash فـ Kali Linux، وفنفس الجلسة (Session) باش المتغيرات اللي كريتي يبقاو شادين. نفّذ خطوة وحدة وحدة، واقرا مزيان شنو طلع فالتيرمينال، وسجّل شنو لقيتي وشنو ما لقيتيش، عاد دوز للأمر اللي من بعدو مورا ما تتأكد من المتطلبات ديالو. ختار أقسام التيكنولوجيا المناسبة للهدف؛ ماشي ضروري تخدم بجميع الأدوات على نفس الموقع. كل بلوك من الكود كيمثل أمراً واحداً. وخا يكون كيبان طويل على بزاف ديال السطور، فـ كتل JSON وYAML انسخ البلوك كامل بما فيه سطر النهاية. ملي يوقع شي إيرور، عالج السبب ديالو قبل ما تكمل. إلا بغيتي توقف الأداة، استعمل Ctrl+C أو زر Stop. راجع البروكسي والفيشيات المحفوظة، والتزم بالحد الأدنى ديال الطلبات؛ وخدم بذكاء اصطناعي محلي غير باش تعاون فالتحليل بلا ما تصيفط ليه أسرار التوثيق والبيانات الحساسة.

▪ كتختار طريق المراجعة على حساب التيكنولوجي اللي لقيتي

قبل ما تبدا الفحص المتقدم، كتشوف السيت فاش مخدوم وكتحدد المسار المناسب كيف موضح فالجدول:

التيكنولوجيا / الطبقة	طريق الفحص والتدقيق
تطبيق ويب	استجابات HTTP، الطلبات الفعلية، الجلسات والصلاحيات، ثم الفحص المحدد.
`API`	عقد الطلب والاستجابة، التوثيق والملكية، Postman أو Burpg Newman.
`WordPress`	التحقق من التقنية، WPScan، ثم شروط الثغرة في الإضافة أو القالب.
`Vercel`	وظائف التطبيق وAPI وقرائن التخزين المؤقت؛ الاستضافة لا تحدد نوع الثغرة.
`Cloudflare`	التمييز بين رد التطبيق وتحديات الحماية وطبقة CDN؛ لا توسيع لنطاق عنوان مشترك.



الشكل 1: دورة حياة اختبار الويب من التيرمينال حتى لتقرير بروفيسيونيل قابل للمراجعة والتدقيق

02 كيفاش تقرا السينطاكس والقيم التوضيحية SYNTAX & VALUE INTERPRETATION

جميع الأمثلة تستخدم عناوين وأسماء ملفات وقيماً محددة. بعض الرموز، مثل متغيرات Nucleig Postmang Bash، جزء من صيغة الأداة ويجب الإبقاء عليها؛ يوضح الجدول معناها. القيمة التوضيحية المكتملة لا تثبت أن الخدمة أو الملف المذكور موجود.

السينطاكس / الرمز	المعنى التطبيقي ديالو
`https://example.com`	عنوان كامل يتضمن البروتوكول واسم المضيف؛ يستخدم للتوثيق.
`127.0.0.1`	الجهاز أو الحاوية أو البيئة التي تنفذ الأمر فيها؛ لا يشير إلى جهاز افتراضي آخر.
`#`	في بداية أسطر أمثلة المخاطر، تجعل السطر تعليقاً غير منفذ.
`"$WEB_URL"`	قراءة متغير Bash داخل تنصيص مزدوج؛ تبقى علامة $ كما هي.
`{{baseUrl}}`	متغير Postman تُحدد قيمته في المجموعة أو عند التشغيل.
`{{BaseURL}}`	متغير Nuclei؛ تختلف حالة حروفه عن متغير Postman.
`FUZZ`	موضع إدراج قيم القائمة داخل طلب ffuf.
`%{http_code}`	صيغة curl لكتابة كود HTTP بعد الطلب.
`|`	أنبوب بين أوامر Bash؛ وقد يكون له معنى آخر داخل تعبير مقتبس خاص بـjq أو grep.
`> reports/result.txt`	كتابة الخرج في الملف المحدد، مع استبدال محتواه السابق إن وجد.
`\`	في نهاية السطر يواصل الأمر في السطر التالي؛ لا يوضع فراغ بعده.
`"text"` / `'text'`	علامات التنصيص تحافظ على تجميع النص؛ تسمح المزدوجة بتوسيع متغيرات Bash.
`-r` / `-p`	حروف خيارات يتغير معناها بحسب الأداة؛ يشرحها جدول كل أمر في سياقها.
`sudo`	صلاحيات محلية مرتفعة للأمر التالي؛ لا يمنح إذناً على موقع بعيد.

في مثال تنزيل السكربت بالأمر 54، الرابط مكتوب بصيغته الكاملة دون أقواس زاوية. إضافة علامتي أصغر من وأكبر من حول اسم النطاق تغيّر تفسير Bash لأنها رموز لإعادة توجيه الإدخال والإخراج، وليست زينة تحيط بالعناوين.

▪ تفهم الأوتـپوت (Output) والستاتوس كود قبل ما تحكم عليه

جميع الأمثلة كتخدم بقيم وعناوين واضحة. بعض الرموز بحال متغيرات Nucleig Postmang Bash هما جزء من الأداة وخاصك تخليهم كيف ما هما. القيمة التوضيحية ما كتعنيش أن الملف كاين بالضرورة. فهم الستاتوس كود والأوتـپوت بهاد المعايير:

الستاتوس كود / المخرج	شنو خاصك تفحص وتراجع
`200`	جسم الرد والهوية وملكية المورد؛ النجاح البروتوكولي وحده لا يثبت تجاوزاً.
`301 / 302`	قيمة Location ونطاق الوجهة قبل متابعة التحويل.
`401 / 403`	هل صدر الرفض من التطبيق أم الوكيل أو WAF؟ وما السلوك المتوقع؟
`404`	هل المورد غير موجود أم يخفيه التطبيق بسبب الصلاحيات؟
`429`	حد الطلبات Retry-Afterg إن وجد؛ إيقاف الفحص ومراجعة المعدل.
`500`	خطأ خادم يحتاج سياقاً؛ لا يثبت SQL injection أو تنفيذ أوامر.
تنبيه ماسح	قاعدة المطابقة والطلب والاستجابة وشروط انطباق الثغرة.
ملف فارغ	نجاح التشغيل واكتماله أولاً؛ قد يدل الفراغ على فشل أو عدم تطابق.

03 بلان ديال الخدمة فـ سيمانتين (14 يوم)

TWO-WEEK SPRINT PLAN (14 DAYS)

هادا بلان مقترح ومجرب باش تقسم قراءة وتطبيق الدليل على هدف مصرح به أو فـ لّاب محلي فـ سيمانتين (14 يوم). هاد الأيام هي مراحل تنظيمية، وماشي وعد باكتشاف ثغرات فمدة معينة.

اليوم	المحور والموضوع	النتيجة والأدلة المطلوبة
1	نطاق الاختبار ومساحة العمل	ملف نطاق ومجلدات أدلة منظمة.
2	HTTP والتقنية	رؤوس وجسم وقياسات وفرضيات أولية.
3	Burp Proxy	طلب محفوظ وشهادة محلية وإعداد نطاق صحيح.
4	Burp Repeater	مقارنة طلب مرجعي وتغيير واحد.
5	`Postman`	مجموعة API صغيرة وعقد تحقق واضح.
6	Newman وتقارير API	تحليل نتيجة كل طلب واختبار.
7	Nmapg Niktog ffuf	استخدام الفرع المسموح فقط وتقييم النتائج.
8	قوالب Nuclei	قالب مقروء ومطابقة تمت مراجعتها.
9	WordPress عند انطباقه	مكونات وإصدارات وشروط انطباق التنبيهات.
10	صلاحيات API وCDN	مقارنة حسابين مملوكين وقراءة رؤوس الكاش.
11	sqlmap في المختبر	فهم سجل الكشف وحدود الاستنتاج.
12	ZAP في المختبر	قراءة تنبيه وربطه بطلب ودليل.
13	تحليل المخاطر والتحقق	استبعاد الأوامر غير الملائمة ومراجعة الفرضيات.
14	التقرير والمثال الختامي	أدلة منظمة وتقرير يحدد الأثر وما بقي غير مؤكد.

▪ خريطة الأقسام وتوزيع الأوامر فالدليل

هاد الجدول كيبين ليك الأوامر من 01 حتى لـ 64 وتوزيعها على المحاور التقنية ديال الدليل:

الأوامر	المحور فالوثيقة
01-08	تهيئة مساحة العمل وتوثيق نطاق الاختبار
09-13	قراءة HTTP وبصمة التطبيق من الطرفية
14-19	Burp Suite من التشغيل إلى اختبار طلب ومقارنته
20-26	Newmang Postman لاختبار API وتحليل التقرير
27-33	اكتشاف المسارات وفحص إعدادات الويب
34-37	Nuclei ومراجعة القالب وتحليل التنبيهات
38-42	WordPress وتشغيل WPScan والتحقق من النتائج
43-47	التحقق من صلاحيات API وقراءة ردود CDN
48-53	كشف الثغرات في مختبر محلي
54-62	تحليل الأوامر مرتفعة الخطورة
63-64	من نتيجة الأداة إلى تقرير يمكن مراجعته

01 تقاد البلاصة فين تخدم وتوثق النطاق (Scope) ديال التيست

01 // WORKSPACE SETUP & SCOPE

يفترض هذا الدليل أن الأدوات مثبتة مسبقاً. ننفذ الأوامر في Bash على Kali، أمراً واحداً ثم نحلل نتيجته. الأسماء example.com www.example.comg أمثلة توثيقية؛ هذا الملف لا يمنح إذناً لفحصهما. في العمل الحقيقي نبدل الهدف بأصل مسموح في سياسة البرنامج. وجود موقع في HackerOne وحده لا يبيح كل أدوات الفحص. مراجع القسم: [1] [2]

الأمر 01: صاوب الدوسي ديال التارغت (Target)

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ mkdir -p ~/bug-bounty/example.com
```

شنو كيدير هاد الأمر: كتصاوب بلاصة مجمّعة للملاحظات والريكويستات والتقارير.

البارامیطر / الخيار	شنو كيعني وكيدير
`mkdir`	إنشاء مجلد.
`-p`	إنشاء المجلدات الأب عند الحاجة؛ لا يفشل إذا كان المجلد موجوداً.
`~`	مجلد المستخدم الحالي.
`bug-bounty/example.com`	مسار محلي لتنظيم ملفات الهدف؛ لا يرسل طلباً للموقع.

النتيجة اللي كتسناها: فالعادة ما كيطلع حتى تيكست فالتيرمينال إلا داز الأمر بنجاح.

تحليل النتيجة وكيفاش تقراها: Permission denied كتعني عندك مشكل فصلاحيات الكتابة فالسيستيم. بدّل المسار لبلاصة كتحكم فيها، وما تحلش المشكل بأنك تزيد sudo على أي حاجة!

الأمر 02: دخل للدوسي ديال الخدمة

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ cd ~/bug-bounty/example.com
```

شنو كيدير هاد الأمر: جعل الملفات التالية داخل مجلد الهدف.

البارامیطر / الخيار	شنو كيعني وكيدير
`cd`	تغيير المجلد الحالي.
bug-bounty/example.com/~	المجلد الذي أنشأناه.

النتيجة اللي كتسناها: يتغير المسار في موجه Terminal غالباً.

تحليل النتيجة وكيفاش تقراها: إذا ظهر No such file or directory، أصلح المسار قبل المتابعة.

الأمر 03: تأكد من البلاصة فين راك كاين فالتيرمينال (pwd)

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ pwd
```

شنو كيدير هاد الأمر: إظهار المسار الكامل.

البارامیطر / الخيار	شنو كيعني وكيدير
`pwd`	اختصار print working directory؛ اطبع مجلد العمل.

النتيجة اللي كتسناها: /home/kali/bug-bounty/example.com

تحليل النتيجة وكيفاش تقراها: المسار المعروض يفترض أن اسم المستخدم المحلي هو kali. يتغير هذا الجزء بحسب حسابك، بينما يظل مجلد الهدف example.com كما أنشأته. تحقق من المجلد الحالي قبل حفظ الأدلة.

الأمر 04: قفل الصلاحيات باش ما يشوف حتى واحد الفيشيات الجداد (umask)

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ umask 077
```

شنو كيدير هاد الأمر: تقليل وصول مستخدمين محليين آخرين إلى الأدلة التي سننشئها.

البارامیطر / الخيار	شنو كيعني وكيدير
`umask`	تحديد الأذونات التي تُحجب عن الملفات الجديدة.
`077`	منع صلاحيات المجموعة والآخرين افتراضياً.

النتيجة اللي كتسناها: لا يظهر نص؛ الإعداد يخص الجلسة الحالية.

تحليل النتيجة وكيفاش تقراها: هذا ليس تشفيراً ولا يغير أذونات الملفات القديمة. تقارير البروكسي وAPI قد تحتوي رموز دخول.

الأمر 05: صاوب الدواسى ديال الأدلة والنوطات والتقارير

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ mkdir -p raw reports notes requests templates
```

شنو كيدير هاد الأمر: فصل الاستجابات الأصلية عن التقارير والقوالب.

البارامیطر / الخيار	شنو كيعني وكيدير
mkdir -p	إنشاء مجلدات دون مسح الموجود.
`raw`	الاستجابات الخام.
`reports`	مخرجات الأدوات.
`notes`	النطاق والتحليل.
`requests`	طلبات API وملفاتها.
`templates`	قوالب Nuclei التي راجعناها.

النتيجة اللي كتسناها: خمسة مجلدات داخل مجلد الهدف.

تحليل النتيجة وكيفاش تقراها: إعادة أمر يحفظ إلى اسم ملف موجود قد تستبدل نتيجته القديمة. استخدم مجلد جلسة جديداً في الاختبارات المتكررة.

الأمر 06: حط لادريس ديال السيت فـ Variable ديال Bash

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ WEB_URL='https://example.com'
```

شنو كيدير هاد الأمر: تحديد أصل الويب الذي ستستعمله أمثلة الأوامر.

البارامیطر / الخيار	شنو كيعني وكيدير
`WEB_URL`	اسم متغير حساس لحالة الحروف.
=	إسناد دون مسافات حول علامة المساواة.
`'https://example.com'`	قيمة نصية؛ HTTPS هو البروتوكول وexample.com المضيف.

النتيجة اللي كتسناها: لا يظهر نص ولا يتم الاتصال.

تحليل النتيجة وكيفاش تقراها: تغيير هذا المتغير لا يغير النطاق القانوني. ولا يغير العناوين الثابتة داخل collection أو قالب أو ملف طلب.

الأمر 07: تآفيشي المتغير وتتأكد منو قبل ما تصيفط أي ريكويست

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ printf 'WEB_URL=%s \
      n' "$WEB_URL"
```

شنو كيدير هاد الأمر: كشف متغير فارغ أو عنوان مكتوب خطأ قبل تشغيل الفحص.

البارامیطر / الخيار	شنو كيعني وكيدير
`printf`	طباعة بتنسيق محدد.
`%s`	مكان إدراج قيمة نصية.
`\n`	سطر جديد.
`"$WEB_URL"`	قراءة المتغير مع الحفاظ على قيمته كوسيط واحد.

النتيجة اللي كتسناها:
WEB_URL=https://example.com

تحليل النتيجة وكيفاش تقراها: إن كان فارغاً، أعد خطوة الإسناد في نفس الجلسة. المتغير لا ينتقل تلقائياً إلى Terminal آخر.

الأمر 08: قيد الحدود والسكوب (Scope) فـ فيشي نوطات

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ nano notes/scope.md
```

شنو كيدير هاد الأمر: فتح ملف نصي لتوثيق ما يسمح به البرنامج.

البارامیطر / الخيار	شنو كيعني وكيدير
`nano`	محرر نصوص تفاعلي.
`notes/scope.md`	ملف الملاحظات الذي سنحفظه.

النتيجة اللي كتسناها: محرر؛ احفظ بـ Ctrl+O ثم Enter واخرج بـ Ctrl+X.

تحليل النتيجة وكيفاش تقراها: اكتب الأصل والمسارات المستثناة وحد الطلبات والحسابات المسموحة وهل fuzzingg scanning مسموحان. إذا كان البرنامج يطلب رأس تعريف للباحث، أضفه في كل أداة قبل استعمالها؛ الأوامر هنا لا تفترض اسمه.

02 قراءة ريكويستات HTTP وبصمة السيت (Fingerprint) من التيرمينال

02 // CLI HTTP PROBING & FINGERPRINTING

ابدأ باستجابة محفوظة يمكن تحليلها محلياً. الرمز 200 HTTP يدل على استجابة ناجحة على مستوى البروتوكول، و403 يدل على رفض الطلب، و500 يدل على خطأ في الخادم. لا يثبت أي منها بمفرده وجود ثغرة أو غيابها. الأوامر المحلية في هذا القسم تعيد قراءة الأدلة دون إرسال طلبات إضافية. مراجع القسم: [3] [6] [31] [32]

الأمر 09: تسجل ريسبونس وميتريكس ديال أول ريكويست بـ curl

سياق الاستخدام: طلب HTTP واحد

```
$ curl --disable --silent --show-error \
      --connect-timeout 5 --max-time 15 \
      --dump-header raw/root.headers \
      --output raw/root.html \
      --write-out 'status=%{http_code} type=%{content_type} seconds=%{time_t
  otal} ' "$WEB_URL/" > raw/root.metrics
```

شنو كيدير هاد الأمر: فصل الرؤوس والجسم وقياسات الاتصال في ثلاثة ملفات.

الباراميطر / الخيار	شنو كيعني وكيدير
`curl`	عميل HTTP؛ الطلب هنا GET.
`--disable`	تجاهل إعدادات curlrc؛ يوضع أول خيار.
--silent --show-error	إخفاء شريط التقدم مع إبقاء الأخطاء.
--connect-timeout 5	مهلة إنشاء الاتصال خمس ثوان.
--max-time 15	مهلة الطلب كله خمس عشرة ثانية.
`--dump-header`	حفظ رؤوس الاستجابة.
`--output`	حفظ جسم الصفحة.
`--write-out`	طباعة الكود والنوع والمدة بعد الطلب.
> raw/root.metrics	توجيه تلك القياسات إلى ملف؛ يستبدل محتواه إن وجد.
`\`	متابعة نفس الأمر في السطر التالي؛ لا تضف مسافة بعدها.

النتيجة اللي كتسناها: root.headers root.htmlg وroot.metricsg؛ القيمة status تختلف حسب الاستجابة.

تحليل النتيجة وكيفاش تقراها: لا يوجد -L، لذلك لا نتبع التحويل آلياً. عند 3xx نراجع Location ونطاق الوجهة. 000 أو ملف ناقص مع خطأ curl مشكلة اتصال أو TLS، وليس غياب ثغرات.

الأمر 10: قرا النتيجة ديال الريكويست والميتريكس

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ cat raw/root.metrics
```

شنو كيدير هاد الأمر: عرض ملخص الطلب السابق دون اتصال جديد.

البارامیطر / الخيار	شنو كيعني وكيدير
`cat`	عرض محتوى ملف نصي.
`raw/root.metrics`	ملف القياسات الذي كتبه curl.

النتيجة اللي كتسناها: مثال توضيحي: status=200 seconds=0.250g type=text/htmlg.

تحليل النتيجة وكيفاش تقراها: الزمن يشمل الشبكة والخادم؛ بطء واحد لا يثبت ثغرة time based. HTML طبيعي للصفحة الرئيسية، لكن HTML بدل JSON في API قد يعني صفحة دخول أو تحدياً من WAF.

الأمر 11: تفحص وتفلي الهيدرز (Headers) الأصليين ديال السيت

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ sed -n '1,80p' raw/root.headers
```

شنو كيدير هاد الأمر: عرض أول ثمانين سطراً من الرؤوس.

البارامیطر / الخيار	شنو كيعني وكيدير
`sed`	معالج نصوص.
`-n`	عدم طباعة كل شيء تلقائياً.
'1,80p'	اطبع السطور من 1 إلى 80.
`raw/root.headers`	مصدر البيانات المحلي.

النتيجة اللي كتسناها: سطر HTTP ورؤوس مثل content-type locationg set-cookieg.

تحليل النتيجة وكيفاش تقراها: حلل Secure HttpOnlyg SameSiteg بحسب وظيفة كل cookie. غياب رأس أمني قد يكون ملاحظة تحسين فقط؛ اشرح أثره القابل للإثبات قبل تسمية النتيجة ثغرة.

الأمر 12: تعرّف على التيكنولوجي من كود الـ HTML المسجل

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ whatweb --color=never --log-json=reports/whatweb.json \
      raw/root.html
```

شنو كيدير هاد الأمر: استعمال WhatWeb على ملف محلي دون مسح الموقع.

الباراميطر / الخيار	شنو كيعني وكيدير
`whatweb`	مطابقة علامات التقنيات.
color=never--	نص دون أكواد تلوين.
log--- json=reports/whatweb.json	تقرير JSON.
`raw/root.html`	ملف محلي وليس عنواناً للشبكة.

النتيجة اللي كتسناها: علامات تقنية إن كانت موجودة في HTML.

تحليل النتيجة وكيفاش تقراها: قد تفوت تقنيات تظهر في الرؤوس أو الصفحات الداخلية. كلمة WordPress أو Next.js قرينة؛ لا تفترض إصداراً أو ثغرة من اسم التقنية.

الأمر 13: فرّز نوع الاستضافة والحماية (Cloudflare / WAF)

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ grep -niE \
      'server:|x-vercel-|cf-ray:|cf-mitigated:|cf-cache-status:' \
      raw/root.headers
```

شنو كيدير هاد الأمر: البحث عن قرائن Vercel وCloudflare داخل الدليل المحفوظ.

الباراميطر / الخيار	شنو كيعني وكيدير
`grep`	البحث في النص.
`-n`	إظهار رقم السطر.
`-i`	تجاهل حالة الحروف.
`-E`	استعمال تعبير منتظم.
`|`	داخل النمط تعني «أو»؛ ليست pipe بين أمرين هنا.

النتيجة اللي كتسناها: رؤوس مطابقة، أو لا شيء.

تحليل النتيجة وكيفاش تقراها: cf-mitigated: challenge يشير لتحدي حماية. x-vercel-cache: HIT وصف كاش. Cloudflare وVercel طبقات خدمة، ويمكن أن تجتمعا مع WordPress أو تطبيق API. لا تثبت هذه الرؤوس تجاوزاً أو ثغرة.

03 // BURP SUITE PROXY & REPEATER

03 Burp Suite: من الديماراج حتى لتيست الريكويست والمقارنة ديالها

Burp واجهة وبروكسي وأدوات تحليل. أمر التشغيل لا يفحص النطاق تلقائياً. Community تكفي لاعتراض الطلبات وRepeater؛ Scanner الآلي من خصائص Professional. ابدأ مشروعاً مؤقتاً أو ملف مشروع متاحاً في إصدارك، وأبقِ الإضافات غير المراجعة معطلة. مراجع القسم: [7] [8] [9] [10] [11]

الأمر 14: ديماري Burp Suite فـ Kali Linux

سياق الاستخدام: واجهة محلية

```
$ burpsuite
```

شنو كيدير هاد الأمر: فتح التطبيق المثبت مسبقاً.

الباراميطر / الخيار	شنو كيعني وكيدير
`burpsuite`	مشغل حزمة Kali؛ لا نضيف نطاق كوسيط وكأنه ماسح سطر أوامر.

النتيجة اللي كتسناها: نافذة Burp واختيار المشروع.

تحليل النتيجة وكيفاش تقراها: في Target ثم Scope أضف https://example.com كأصل المثال بالضبط. راجع المضيف والمنفذ والمسارات؛ لا توسع النطاق إلى النطاقات الفرعية آلياً. فلتر Show only in scope ينظم العرض، ولا يمنع كل الطلبات الخارجة وحده.

الأمر 15: تشيكي واش البروكسي خدام على 127.0.0.1:8080

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ ss -ltn 'sport = :8080'
```

شنو كيدير هاد الأمر: التأكد أن هناك خدمة TCP تستمع على منفذ Burp الافتراضي.

الباراميطر / الخيار	شنو كيعني وكيدير
`ss`	عرض مقابس الشبكة.
`-l`	منافذ تستمع.
`-t`	`TCP.`
`-n`	إظهار الأرقام دون تحويل أسماء.
sport = :8080	تصفية المنفذ المحلي 8080.

النتيجة اللي كتسناها: المتوقع Listener على 127.0.0.1:8080 إذا كانت إعدادات Burp الافتراضية مفعلة.

تحليل النتيجة وكيفاش تقراها: هذا لا يثبت أن الخدمة هي Burp؛ راجع Proxy listeners في Burp. احصر الاستماع في واجهة loopback المحلية. إذا كان المنفذ مستعملاً من أداة أخرى اختر منفذاً محلياً آخر وغيّر الأوامر المرتبطة.

الأمر 16: التيليشارجي السيرتيفيكا (CA Cert) ديال Burp

سياق الاستخدام: اتصال محلي

```
$ curl --disable --silent --show-error \
      --proxy http://127.0.0.1:8080 --noproxy "" \
      --max-time 10 --output raw/burp-ca.der \
      http://burp/cert
```

شنو كيدير هاد الأمر: الحصول على CA الخاصة بنسخة Burp المحلية لتوثيق اتصال curl بها.

البارامیطر / الخيار	شنو كيعني وكيدير
`--proxy`	توجيه الطلب إلى البروكسي المحلي.
--noproxy ""	تجاوز قائمة استثناءات البروكسي للجلسة.
`http://burp/cert`	عنوان خاص يجيب عنه Burp.
--output raw/burp-ca.der	حفظ الشهادة العامة بصيغة DER.
--max-time 10	حد زمني للطلب المحلي.
--disable --silent --show-error	ضبط curl كما سبق.

النتيجة اللي كتسناها: ملف شهادة؛ يجب أن يكون Burp قيد التشغيل وIntercept off لهذا الطلب.

تحليل النتيجة وكيفاش تقراها: لا نصدّر المفتاح الخاص ولا نثق بشهادة عشوائية من الإنترنت. فشل التحويل في الخطوة التالية قد يعني أنك حفظت صفحة خطأ بدل شهادة.

الأمر 17: تحول السيرتيفيكا لـ صيغة PEM باش يقبلها curl

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ openssl x509 -inform DER -in raw/burp-ca.der \
      -out raw/burp-ca.pem
```

شنو كيدير هاد الأمر: تجهيز الشهادة بصيغة يقبلها curl.

البارامیطر / الخيار	شنو كيعني وكيدير
openssl x509	قراءة وتحويل شهادة X.509.
-inform DER	صيغة الملف الداخل.
`-in`	ملف الإدخال.
`-out`	ملف الإخراج؛ PEM افتراضياً.

النتيجة اللي كتسناها: raw/burp-ca.pem عند النجاح.

تحليل النتيجة وكيفاش تقراها: لا تضف -k عند الخطأ كي تخفي المشكلة؛ تحقق من الشهادة والمنفذ ونسخة Burp التي أصدرتها.

الأمر 18: تصيفط ريكويست دايزة من وسط Burp Suite

سياق الاستخدام: طلب عبر البروكسي

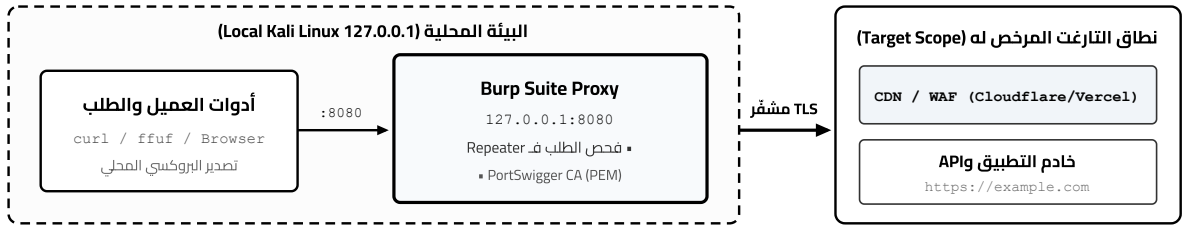
```
$ curl --disable --silent --show-error \
      --proxy http://127.0.0.1:8080 --noproxy "" \
      --cacert raw/burp-ca.pem --max-time 15 \
      --dump-header raw/burp.headers \
      --output raw/burp.body "$WEB_URL/"
```

شنو كيدير هاد الأمر: رؤية نفس طلب curl داخل Proxy ثم HTTP history.

البارامیطر / الخيار	شنو كيعني وكيدير
--proxy --noproxy	استعمال Burp حتى لو كان NO_PROXY مضبوطاً.
`--cacert`	الثقة بشهادة Burp لهذا الأمر فقط.
--max-time 15	مهلة الطلب.
--dump-header --output	حفظ الرؤوس والجسم.
`"$WEB_URL/"`	جذر الموقع المحدد.

النتيجة اللي كتسناها: طلب واحد في HTTP history وملفان محليان.

تحليل النتيجة وكيفاش تقراها: Intercept on يوقف الطلب في Burp إلى أن تضغط Forward؛ المهلة حينها لا تعني أن الخادم بطيء. curl يوثق TLS إلى Burp؛ TLS بين Burp والخادم تضبطه إعدادات Burp.



الشكل 2: بنية بيئة الاختبار وتوجيه الترافيك عبر وسيط Burp Suite المحلي (127.0.0.1:8080)

▪ تيست مانويل (يدوي) فـ Burp Repeater

من الـ HTTP history ديال Burp، اختار الريكويست اللي مسموح تفحصها وكليكي Send to Repeater. فـ Settings ديال Repeater، حيد تفعيل Follow redirects (دير Never). كليكي Send أول مرة باش تسجل ريسبونس مرجعي أصلي. من بعد، بدّل بارامیطر واحد فكل مرة، بحال إلا حيدتي الهيدر ديال Authorization من ريكويست GET باش تشوف واش الباك إند كيعزل الصلاحيات ولا كيجاوب بالداتا بلا مصادقة.

▪ ترجع الدليل والترافيك للتيرمينال

من محرر الرسائل فـ Burp، حفظ البودي الأصلي بـ Copy to file ولا خيار Save message اللي عندك. سمي ريسبونس المرجع raw/baseline.body والريسبونس اللي بدلتيه raw/changed.body. إلا سجلتي ريكويست HTTP كاملة بالهيدرز، فرّق الهيدرز على البودي قبل ما دوز لـ jq باش ما يتبلونطاش ليك التحليل.

الأمر 19: دير مقارنة (diff) بين جوج ريسبونسات

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ diff -u raw/baseline.body raw/changed.body
```

شنو كيدير هاد الأمر: عرض ما تغير بعد اختبار Repeater المنضبط.

البارامیطر / الخيار	شنو كيعني وكيدير
`diff`	مقارنة ملفين.
`-u`	عرض فرق موحد مع سياق.
`baseline.body`	الاستجابة المرجعية.
`changed.body`	استجابة تغيير واحد.

النتيجة اللي كتسناها: سطور تبدأ بـ - للحذف و+ للإضافة؛ أو لا اختلاف.

تحليل النتيجة وكيفاش تقراها: خروج diff بالرمز 1 يعني اختلاف الملفات، وليس فشل الأداة. تواريخ وCSRF tokens قد تتغير طبيعياً. الدليل الأمني هو الوصول غير المسموح أو الأثر، لا مجرد اختلاف الحجم.

▪ الفحص الأوتوماتيكي فـ Burp Professional (Audit)

إلا عندك نسخة Professional، حدد الريكويست المصرح بها من السجل ودير Scan أو Audit selected items على حساب النسخة. اختار ديما تدقيق الطلب المحدد (Audit) بلا ما تطلق كراولر (Crawler) واسع يضيع ليك الوقت. راجع إعدادات الفحص وحوض الموارد (resource pool) واضبط ريكويست وحدة فالثانية مع ديلاي كيحترم سياسة البرنامج، وحبس السكان فالبلاصة إلا حسيتي بالسيرفور ثقال.

04 Newmang Postman لاختبار الـ API وقراءة التقارير

04 // API TESTING (POSTMAN & NEWMAN)

Postman Desktop واجهة، وPostman CLI أمر لتشغيل collections، Newmang مشغل مستقل لمجموعات v2.1 JSON. لا نفترض أن example.com يقدم API. المسار /api/account أدناه مثال لعقد API يقول إن الطلب دون تسجيل دخول يجب رفضه؛ استبدله بمسار حقيقي معروف ومسموح. لا ترسل collection غير مراجعة: قد تحوي scripts وطلبات إضافية. مراجع القسم: [12] [13] [14] [15] [16] [17]

الأمر 20: ديماري لانترفاس ديال Postman

سياق الاستخدام: واجهة محلية

```
$ snap run postman
```

شنو كيدير هاد الأمر: فتح نسخة Desktop إذا كانت مثبتة مسبقاً بصيغة Snap.

البارامیطر / الخيار	شنو كيعني وكيدير
snap run	تشغيل تطبيق موجود؛ ليس تثبيتاً.
`postman`	اسم تطبيق Snap.

النتيجة اللي كتسناها: نافذة Postman على جلسة Linux رسومية.

تحليل النتيجة وكيفاش تقراها: هذا الأمر خاص بنسخة Snap. إن كانت لديك نسخة مستقلة شغّل ملف Postman التنفيذي من مساره الموجود أو من قائمة التطبيقات. الاسم postman قد يشير إلى CLI عندك، لذلك لا نخلط المشغلين ولا نستعمل sudo.

- **توجد الريكويست من لانترفاس ديال Postman**

كريي Collection جديدة سميها API authorization. زيد ريكويست GET بعنوان {{baseUrl}}/api/account. حط المتغير baseUrl كـ https://example.com فـ Collection variables، وحيد تفعيل Automatically follow redirects من إعدادات الطلب. خلي Authorization على No Auth إلا كنت باغي تجرب الطلب بلا مصادقة وتشوف رد الخادم.

- **تسجل الكوليكشن بلا ما تخرّق فالصيغة ديال الفيشي**

تقدر تدير Export للكوليكشن لـ requests/api.postman_collection.json بصيغة JSON v2.1 إلا كانت متوفرة عندك فـ Postman. ركز مزيان حيت Newman ما كيشغلش صيغة v3 الجديدة. الخطوة الموالية كتعلمك كيفاش تصاوب فيشي v2.1 صغير ومكتمل محلياً بلا تعقيد.

الأمر 21: تصاوب كوليكشن API صغيرة ومحكومة فـ Postman

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ cat > requests/api.postman_collection.json { "info": { "name": "API author
  ization - unauthenticated request", "schema": "https://schema.getpostman.c
  om/json/collection/v2.1.0/collection.json" }, "variable": [ {"key": "baseU
  rl", "value": "https://example.com"} ], "item": [{ "name": "Account requir
  es authentication", "request": { "method": "GET", "auth": {"type": "noaut
  h"}, "url": "{{baseUrl}}/api/account" }, "event": [{ "listen": "test", "sc
  ript": { "type": "text/javascript", "exec": [ "pm.test('Unauthenticated re
  quest is rejected',", "function () {", " pm.expect(pm.response.code).to.b
  e.oneOf([401, 403]);", "});" ] } }] }] } JSON
```

شنو كيدير هاد الأمر: كتابة collection v2.1 فيها GET واحد واختبار يطابق عقد منع المستخدم غير المسجل.

البارامیطر / الخيار	شنو كيعني وكيدير
cat >	كتابة النص حتى سطر JSON الأخير؛ الاقتباس يمنع توسيع متغيرات Bash.
info / schema	اسم المجموعة وصيغتها.
variable / baseUrl	الأصل الذي ستركب عليه الطلبات.
item / request	الطلب، طريقته وعنوانه.
`noauth`	لا يضيف token تلقائياً.
event / listen: test	اختبار JavaScript ينفذ بعد الرد.
`pm.test`	اسم الاختبار ودالته.
`pm.response.code`	كود HTTP الفعلي.
oneOf([401, 403])	نجاح إذا كان الرد أحد الكودين.

النتيجة اللي كتسناها: ملف JSON محلي؛ لا يتم إرسال أي طلب عند إنشائه.

تحليل النتيجة وكيفاش تقراها: 401 أو 403 قد يأتي من WAF، لذلك نجاح assertion لا يثبت أن صلاحيات التطبيق سليمة. 404 قد يعني أن مسار المثال غير موجود، و200 قد يكون صفحة دخول. حلل المحتوى والسياق قبل الحكم.

الأمر 22: تفحص محتوى الكوليكشن قبل ما تلونصيها

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ jq . requests/api.postman_collection.json
```

شنو كيدير هاد الأمر: التأكد من صحة JSON وقراءة الطلبات والـscripts.

البارامیطر / الخيار	شنو كيعني وكيدير
`jq`	معالج JSON.
`.`	عرض الكائن كاملاً بتنسيق مقروء.
requests/api.postman_collection.json	المجموعة المحلية.

النتيجة اللي كتسناها: JSON منسق، أو parse error عند كسر الصياغة.

تحليل النتيجة وكيفاش تقراها: راجع كل URL وmethod وكل script قبل التشغيل. صيغة JSON الصحيحة لا تمنع طلب DELETE ولا pm.sendRequest إلى عنوان آخر. لا ترفع أسرار البرنامج إلى مساحة سحابية بلا إذن.

الأمر 23: تراني الكوليكشن بـ Postman CLI

سياق الاستخدام: تشغيل API مشروط بوجود المسار

```
$ postman collection run requests/api.postman_collection.json \
      --env-var "baseUrl=https://example.com" \
      -n 1 --delay-request 1000 --timeout-request 10000 \
      --ignore-redirects --bail failure \
      -r cli,json --reporter-json-structure newman \
      --reporter-json-export reports/postman.json
```

شنو كيدير هاد الأمر: تشغيل طلب المجموعة وإنتاج تقرير محلي قابل للتحليل.

البارامیطر / الخيار	شنو كيعني وكيدير
postman collection run	تشغيل collection بواسطة CLI؛ يتطلب وجود CLI نفسه.
`--env-var`	تعيين متغير بيئة لهذه الجولة؛ يتقدم على متغير collection.
n 1-	جولة واحدة.
delay-request 1000--	انتظار 1000 ميلي ثانية بين الطلبات.
timeout-request 10000--	مهلة الطلب عشرة آلاف ميلي ثانية.
`--ignore-redirects`	عدم تتبع 3xx.
bail failure--	التوقف بعد فشل اختبار.
r cli,json-	إخراج إلى الشاشة وJSON.
reporter-json-structure-- newman	اختيار بنية التقرير المتوافقة مع أوامر وز التالية.
`--reporter-json-export`	مسار ملف التقرير.

النتيجة اللي كتسناها: اسم الاختبار ونجاحه أو فشله وتقرير reports/postman.json.

تحليل النتيجة وكيفاش تقراها: المجموعة المحلية غير المرتبطة بـPostman لا ترفع نتائجها وفق التوثيق؛ أما تشغيل موارد مرتبطة بحساب فيختلف. افحص إعداداتك ورسالة CLI. إذا كان الخيار غير معروف في إصدارك، لا تحذفه بصمت: استخدم بديل Newman الموثق أدناه مع ملف v2.1.

الأمر 24: بديل محلي: تشغيل نفس الكوليكشن بـ Newman فالتيرمينال

سياق الاستخدام: بديل للخطوة السابقة

```
$ newman run requests/api.postman_collection.json \
      --env-var "baseUrl=https://example.com" \
      -n 1 --delay-request 1000 --timeout-request 10000 \
      --ignore-redirects --bail failure \
      -r cli,json --reporter-json-export reports/newman.json
```

شنو كيدير هاد الأمر: تشغيل ملف v2.1 دون الاعتماد على Postman CLI.

البارامیطر / الخيار	شنو كيعني وكيدير
newman run	تشغيل مجموعة Postman v2.1 JSON.
env-var / -n--	تعيين الأصل وجولة واحدة.
delay-request / ---- timeout-request	تأخير ومهلة بالميلي ثانية.
ignore-redirects / --bail-- failure	منع متابعة التحويل والتوقف عند فشل assertion.
r cli,json-	عرض وتصدير.
`--reporter-json-export`	اسم تقرير Newman.

النتيجة اللي كتسناها: reports/newman.json؛ لا تشغل البديلين لمجرد التكرار.

تحليل النتيجة وكيفاش تقراها: هذه مقارنة مشغلات وليست فحصين أمنيين مختلفين. Exit code غير صفري قد ينتج عن assertion أو شبكة أو ملف مكسور؛ اقرأ التفاصيل.

الأمر 25: قراءة رابور Newman وملخص نجاح وفشل الريكويستات

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ jq '{stats: .run.stats, failures: .run.failures}' \
     reports/newman.json
```

شنو كيدير هاد الأمر: استخراج الإحصاءات والأخطاء من تقرير Newman.

البارامیطر / الخيار	شنو كيعني وكيدير
`jq`	قراءة JSON.
stats / failures	بناء ملخص بحقلين.
`.run.stats`	عدادات الطلبات والاختبارات.
`.run.failures`	تفاصيل الاختبارات أو التنفيذ الفاشل.

النتيجة اللي كتسناها: إحصاءات requests وassertionsg وقائمة failures.

تحليل النتيجة وكيفاش تقراها: إذا استخدمت Postman CLI فبدل اسم الملف بـ reports/postman.json. جميع الاختبارات ناجحة تعني فقط أن assertions المكتوبة نجحت؛ لا تعني أن API خالية من الثغرات.

الأمر 26: تربط كل ريكويست بالستاتوس كود ووقت الاستجابة

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ jq -r '.run.executions[] | [.item.name, .response.code, .response.response
  Time] | @tsv' reports/newman.json
```

شنو كيدير هاد الأمر: عرض جدول نصي يسمح بمراجعة النتائج طلباً بطلب.

البارامیطر / الخيار	شنو كيعني وكيدير
`-r`	إخراج نص خام.
`.run.executions[]`	مرور على كل تنفيذ.
`.item.name`	اسم الطلب.
`.response.code`	كود HTTP.
`.response.responseTime`	مدة بالميلي ثانية.
@tsv	حقول مفصولة بعلامة tab.
`|`	هنا مشغل داخل لغة jq لتمرير القيمة للمرحلة التالية.

النتيجة اللي كتسناها: سطر لكل طلب: الاسم ثم الكود ثم المدة.

تحليل النتيجة وكيفاش تقراها: null في response قد يعني أن الطلب لم يحصل على رد. ارجع إلى failures قبل تفسيره كرفض أمني. هذه المجموعة مثال اختبار توقع واحد، وليست ماسح SQLi أو XSS تلقائياً.

05

كشف المسارات المخفية (Fuzzing) وتدقيق إعدادات السيت

05 // DIRECTORY FUZZING & CONFIG AUDITING

ffuf يرسل قيماً من قائمة؛ ليس كل مسار يجده ثغرة. Nikto يبحث عن مؤشرات إعدادات وملفات معروفة. Nmap يفحص منافذ عنوان الشبكة الذي يحل إليه الاسم. اختر الفرع الذي يسمح به نطاقك؛ لا تشغل الجميع بالتوازي، لأن حدود الطلبات تجمع على الهدف نفسه. مراجع القسم: [18] [19] [20] [21] [33]

الأمر 27: تصاوب ليستة ديال المسارات صغيرة ومضبوطة للفازينغ

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ printf '%s \
      n' robots.txt sitemap.xml login api/health \
      > requests/paths.txt
```

شنو كيدير هاد الأمر: تحديد أربع قيم نراجعها بدل قائمة ضخمة مجهولة.

البارامیطر / الخيار	شنو كيعني وكيدير
'printf '%s\n	طباعة كل قيمة في سطر.
robots.txt sitemap.xml login api/health	أربعة مسارات توضيحية، بلا / في البداية.
> requests/paths.txt	حفظ القائمة محلياً.

النتيجة اللي كتسناها: ملف من أربعة أسطر.

تحليل النتيجة وكيفاش تقراها: هذه المسارات لا تعني أنها موجودة على example.com. بدّلها بمسارات تسمح السياسة باستكشافها. robots.txt توجيه للزواحف وليس إذناً لاختبار كل ما يذكره.

الأمر 28: تراني ffuf بريكويست وحدة فالثانية (بلا سبام)

سياق الاستخدام: فحص مسارات مشروط

```
$ ffuf -u 'https://example.com/FUZZ' \
      -w requests/paths.txt -t 1 -rate 1 \
      -timeout 10 -maxtime 60 -mc all \
      -noninteractive -of json -o reports/ffuf.json
```

شنو كيدير هاد الأمر: إرسال GET لكل قيمة في القائمة مع حفظ جميع أكواد HTTP.

البارامیطر / الخيار	شنو كيعني وكيدير
`-u`	عنوان الطلب.
`FUZZ`	مكان استبدال القيمة.
`-w`	ملف القيم.
t 1-	عامل واحد.
rate 1-	سقف طلب واحد في الثانية.
timeout 10-	مهلة كل طلب بالثواني.
maxtime 60-	حد إجمالي بالدقيقة.
mc all-	عرض كل الأكواد، حتى 404.
`-noninteractive`	تعطيل واجهة التحكم التفاعلية.
of json / -o-	صيغة التقرير ومساره.

النتيجة اللي كتسناها: نتيجة لكل مسار مكتمل؛ الكود والحجم وعدد الكلمات.

تحليل النتيجة وكيفاش تقراها: لا نضيف -r لمتابعة التحويل، ولا recursion أو auto calibration لأنها توسع الطلبات. الإعدادات الشخصية في ffufrc قد تغير السلوك؛ راجعها قبل التشغيل. أوقف الفحص عند 429 أو تكرار أخطاء الخادم.

الأمر 29: قراءة نتائج ffuf حبة بحبة وفرز المسارات اللي كاينين

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ jq -r '.results[] | [.url, .status, .length, .words, .lines] | @tsv' repor
  ts/ffuf.json
```

شنو كيدير هاد الأمر: ترتيب النتائج في أعمدة نصية.

البارامیطر / الخيار	شنو كيعني وكيدير
`.results[]`	كل نتيجة محفوظة.
.status. / url	المسار وكود الرد.
length / .words / .lines.	حجم الجسم وعدد الكلمات والسطور.
r / @tsv-	نص مفصول بـtab.

النتيجة اللي كتسناها: أعمدة URL lengthg statusg linesg wordsg.

تحليل النتيجة وكيفاش تقراها: 200 بالحجم نفسه لكل المسارات قد يكون صفحة عامة تسمى soft 404. 403 ليس إثباتاً أن خلفه ملف حساس. قارن الجسم بمرجع لمسار غير موجود قبل تقرير وجود المورد.

الأمر 30: فحص إعدادات وملفات السيت بـ Nikto فحدود السكوب

سياق الاستخدام: فحص نشط مشروط

```
$ nikto -h https://example.com -Tuning 123 \
      -Pause 2 -maxtime 3m -timeout 10 -nocheck \
      -Format txt -output reports/nikto.txt
```

شنو كيدير هاد الأمر: تشغيل فئات ملفات مثيرة للاهتمام وإعدادات وإفشاء معلومات، مع حد زمني.

البارامیطر / الخيار	شنو كيعني وكيدير
`-h`	الهدف.
Tuning 123-	الفئات 1 و2 و3؛ اختيار نوع الفحوص.
Pause 2-	فاصل ثانيتين بين الاختبارات.
maxtime 3m-	ثلاث دقائق لكل مضيف.
timeout 10-	مهلة الطلب عشر ثوان.
`-nocheck`	تجنب فحص التحديث عند البدء.
Format txt / -output-	تقرير نصي ومساره.

النتيجة اللي كتسناها: تنبيهات أو لا تطابقات، وقد ينتهي قبل إكمال قاعدة الاختبارات بسبب الحد الزمني.

تحليل النتيجة وكيفاش تقراها: هذا فحص نشط متعدد الطلبات رغم تخفيض المعدل. Tuning لا يجعل كل plugin آمناً؛ راجع إعدادات Nikto والاختبارات والسياسة. لا نفعّل متابعة التحويل. غياب رأس حماية أو رقم إصدار قديم يحتاج سياقاً وأثراً؛ لا يساوي تنفيذ أوامر.

الأمر 31: قراءة تقرير Nikto وتفريك التنبيهات الملقية

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ sed -n '1,160p' reports/nikto.txt
```

شنو كيدير هاد الأمر: مراجعة بداية التقرير محلياً.

البارامیطر / الخيار	شنو كيعني وكيدير
sed -n	عرض مقيد.
'1,160p'	أول 160 سطراً.
`reports/nikto.txt`	نتائج الأداة.

النتيجة اللي كتسناها: الهدف ووقت الفحص وملاحظاته وأي أخطاء.

تحليل النتيجة وكيفاش تقراها: لكل تنبيه دوّن URL والطلب والدليل. إن كان التقرير مقتطعاً اقرأ بقية الملف في محرر؛ لا تفترض أنه اكتمل ولا تعاود الفحص لمجرد وجود خطأ.

الأمر 32: سكاني بورتات الويب (80 و 443) بـ Nmap

سياق الاستخدام: فحص شبكة مشروط

```
$ nmap -sT -Pn -n -p 80,443 \
      --max-rate 2 --max-retries 1 --host-timeout 60s \
      -oA reports/nmap example.com
```

شنو كيدير هاد الأمر: فحص اتصال TCP إلى منفذي الويب فقط عندما يكون اختبار الشبكة مشمولاً.

البارامیطر / الخيار	شنو كيعني وكيدير
`-sT`	محاولة اتصال TCP كاملة؛ لا تحتاج عادة root.
`-Pn`	تجاوز اكتشاف حياة المضيف وبدء فحص المنافذ.
`-n`	إلغاء البحث العكسي للأسماء؛ ما زال الاسم الهدف يحتاج DNS.
p 80,443-	حصر المنافذ.
max-rate 2--	حد معدل إرسال مجسات Nmap بالثانية.
max-retries 1--	تقليل إعادة المحاولة.
host-timeout 60s--	حد زمني للمضيف.
oA reports/nmap-	حفظ صيغ nmap وXML وgnmap.

النتيجة اللي كتسناها: open أو closed أو filtered لكل منفذ مكتمل.

تحليل النتيجة وكيفاش تقراها: open يعني أن الخدمة تقبل الاتصال؛ لا يثبت ثغرة. filtered قد يكون جداراً نارياً. إذا كان الاسم خلف CDN مشترك فلا تفحص عنوان CDN باعتباره ملك التطبيق. هذا الفرع لا يطبق لمجرد وجود نطاق ويب في scope.

الأمر 33: قراءة البورتات المفتوحين والخدمات اللي شادة فيهم

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ grep -nE '^(PORT|80/tcp|443/tcp)|Nmap done' \
      reports/nmap.nmap
```

شنو كيدير هاد الأمر: استخراج السطور الأساسية من تقرير الشبكة.

الباراميطر / الخيار	شنو كيعني وكيدير
grep -nE	بحث بنمط موسع مع أرقام السطور.
^	بداية السطر.
(PORT|80/tcp|443/tcp)	واحد من العناوين المطلوبة.
Nmap done	ملخص اكتمال Nmap إن وجد.

النتيجة اللي كتسناها: المنافذ والحالة وملخص الجولة.

تحليل النتيجة وكيفاش تقراها: اقرأ التقرير كاملاً إن غاب الملخص أو ظهر timeout. عدم ظهور منفذ في تقرير غير مكتمل لا يثبت إغلاقه.

06 Nuclei: مراجعة القوالب وتحليل النتائج والتنبيهات

06 // NUCLEI TEMPLATES & TRIAGE

تشغيل nuclei على هدف دون اختيار دقيق قد يحمّل مجموعة كبيرة من القوالب. نبدأ بقالب محلي واضح يطلب ملفاً واحداً ويراجع شكل الاستجابة، ثم نراجع كل قالب إضافي قبل تشغيله. درجة severity تصف تصنيف التنبيه، ولا تصف أمان تشغيل القالب. مراجع القسم: [22] [23] [24]

الأمر 34: تكتب قالب Nuclei محدود وموجه لهدف معين

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ cat > templates/git-head.yaml id: diouane-git-head-check info: name: Git H
  EAD response indicator author: DiouaneCyberSecurity severity: info descrip
  tion: Check one exposed metadata file only. http: - method: GET path: - "
  {{BaseURL}}/.git/HEAD" redirects: false matchers-condition: and matchers:
  - type: status status: - 200 - type: regex part: body regex: - '^ref: ref
  s/heads/[A-Za-z0-9._/-]+' YAML
```

شنو كيدير هاد الأمر: فحص مؤشر كشف metadata من Git دون تنزيل المستودع.

البارامیطر / الخيار	شنو كيعني وكيدير
cat >	حفظ نص YAML كما هو.
id / info	هوية القالب ووصفه.
severity: info	تسمية ملاحظة معلوماتية؛ لا ندعي تسريب المصدر.
method: GET	قراءة المسار المحدد.
`{{BaseURL}}`	الأصل الذي تمرره للأداة.
`/.git/HEAD`	ملف metadata واحد.
redirects: false	لا تتبع التحويل.
matchers-condition: and	الشرطان معاً.
status: 200	رد ناجح.
regex / part: body	نمط يشبه مرجع فرع Git داخل الجسم.

النتيجة اللي كتسناها: ملف قالب محلي فقط.

تحليل النتيجة وكيفاش تقراها: الملف قد يحتوي metadata عامة أو تكون الاستجابة تقليداً؛ يلزم تحقق. حتى تأكيد كشف HEAD لا يثبت تسرب كامل الشيفرة. لا نتابع بتنزيل المستودع أو الأسرار ضمن هذا الإجراء.

الأمر 35: تشيكي السينطاكس والصحة ديال قالب Nuclei

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ nuclei -validate -t templates/git-head.yaml -duc
```

شنو كيدير هاد الأمر: اكتشاف أخطاء YAML وبنية القالب قبل استهداف الموقع.

البارامیطر / الخيار	شنو كيعني وكيدير
`-validate`	التحقق من صحة القالب.
`-t`	اختيار ملف واحد.
`-duc`	تعطيل فحص تحديث الأداة والقوالب تلقائياً.

النتيجة اللي كتسناها: نتيجة validation أو رسالة خطأ.

تحليل النتيجة وكيفاش تقراها: نجاح الصيغة لا يعني أن القالب آمن أو أن matchers صحيحة. راجع method وpayloadsg pathg وأي OAST أو code أو headless قبل اعتماد قالب من الإنترنت.

الأمر 36: تراني القالب اللي راجعتيه بمعدل ريكويست وحدة فالثانية

سياق الاستخدام: طلب فحص مشروط

```
$ nuclei -u https://example.com -t templates/git-head.yaml \
      -rl 1 -c 1 -bs 1 -timeout 10 -retries 0 \
      -no-interactsh -disable-redirects -duc \
      -jsonl -o reports/nuclei.jsonl
```

شنو كيدير هاد الأمر: تشغيل القالب المحدد وإخراج التطابقات إلى JSONL.

البارامیطر / الخيار	شنو كيعني وكيدير
u / -t-	هدف واحد وقالب واحد.
rl 1 / -c 1 / -bs 1-	طلب في الثانية، وقالب ومضيف واحدان بالتوازي.
timeout 10 / -retries 0-	مهلة الطلب وعدم إعادته.
`-no-interactsh`	إلغاء OAST الخاص بـInteractsh.
`-disable-redirects`	منع التحويلات في قوالب HTTP.
`-duc`	إلغاء فحص التحديث.
jsonl / -o-	كائن JSON لكل سطر في ملف النتائج.

النتيجة اللي كتسناها: تطابق إن تحققت الشروط، أو ملف فارغ عند عدم التطابق.

تحليل النتيجة وكيفاش تقراها: الملف الفارغ قد يعني عدم التطابق أو فشل اتصال؛ راجع سجل التشغيل. هذا فحص مؤشر واحد فقط. خيارات المعدل لا تحمي من قالب تخريبي؛ لا تستبدل -t بمجلد CVEs كامل ولا تفعّل code أو OAST دون مراجعة واختصاص النطاق.

الأمر 37: قراءة سبب تنبيه Nuclei والماتشينغ اللي وقع

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ jq -r '[."template-id", .info.severity, ."matched-at", ."matcher-name"] |
  @tsv' \
      reports/nuclei.jsonl
```

شنو كيدير هاد الأمر: عرض القالب والتصنيف ومكان التطابق.

البارامیطر / الخيار	شنو كيعني وكيدير
`."template-id"`	هوية القالب؛ الاقتباس لأن الاسم يحوي شرطة.
`.info.severity`	تصنيف القالب.
`."matched-at"`	المكان الذي تطابقت عنده الشروط.
`."matcher-name"`	اسم matcher إن كان مسمى؛ قد يكون null.
@tsv	أعمدة نصية.

النتيجة اللي كتسناها: سطر لكل تطابق؛ severity هنا info في القالب الموضح.

تحليل النتيجة وكيفاش تقراها: راجع الاستجابة الخام المحفوظة إن احتواها التقرير ثم أعد طلباً واحداً في Burp. لا تعِد نشر request أو response يحوي أسراراً؛ استخرج أقل دليل لازم.

07 // WORDPRESS & WPSCAN AUDITING

07 WordPress: تشغيل WPScan والتحقق من النتائج

استعمل هذا الفرع فقط إذا وجدت قرائن WordPress على الهدف المسموح. WPScan يقارن مكونات وإصدارات بمعلومات ثغرات؛ قد يتطلب أثر الثغرة دوراً معيناً أو إعداداً خاصاً. Passive داخل WPScan يقلل طرق الاكتشاف النشطة، لكنه لا يعني انعدام طلبات الشبكة. مراجع القسم: [25] [26] [27] [28] [29]

الأمر 38: تقرا الـ API Token ديال WPScan بلا ما تفضحو فالشاشة

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ read -r -s -p 'WPScan API token: ' WPSCAN_API_TOKEN
```

شنو كيدير هاد الأمر: إدخال رمز API من حساب WPScan المصرح باستخدامه لحفظه مؤقتاً في متغير Bash.

البارامیطر / الخيار	شنو كيعني وكيدير
`read`	قراءة إدخال المستخدم.
`-r`	عدم تفسير الشرطة العكسية.
`-s`	عدم عرض الرمز أثناء الكتابة.
`-p`	رسالة طلب الإدخال.
`WPSCAN_API_TOKEN`	المتغير الذي ستقرأه الأداة بعد تصديره.

النتيجة اللي كتسناها: موجه إدخال؛ ألصق الرمز ثم Enter دون إظهاره في مخرجات الطرفية.

تحليل النتيجة وكيفاش تقراها: النص الذي تدخله هنا ليس أمر shell. لا تدخل الرمز الحقيقي ضمن عرض الشاشة أو المستند أو سجل المشاركة. إذا لم يتوفر رمز API، تخطَّ خطوتي الرمز وافهم أن بيانات الثغرات قد لا تظهر.

الأمر 39: دوز التوكن لـ WPScan فالجلسة الحالية

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ export WPSCAN_API_TOKEN
```

شنو كيدير هاد الأمر: السماح للأداة التي ستبدأ من نفس الجلسة بقراءة المتغير.

البارامیطر / الخيار	شنو كيعني وكيدير
`export`	إتاحة متغير لعمليات الأبناء.
`WPSCAN_API_TOKEN`	اسم الرمز دون عرض قيمته.

النتيجة اللي كتسناها: لا يظهر نص.

تحليل النتيجة وكيفاش تقراها: ينطبق على نفس جلسة Terminal؛ فتح جلسة أخرى لا ينقل القيمة تلقائياً. المتغير في البيئة ليس مخزناً مشفراً. أغلق جلسة العمل عند انتهائها ولا تصدر بيئتها أو تقريراً يحوي الرمز.

الأمر 40: فحص WordPress مع تقييد التعداد (Enumeration)

سياق الاستخدام: فحص WordPress مشروط

```
$ wpscan --url https://example.com \
      --scope example.com \
      --enumerate vp,vt --detection-mode passive \
      --plugins-detection passive \
      --plugins-version-detection passive \
      --themes-detection passive \
      --themes-version-detection passive \
      --throttle 1000 --request-timeout 10 \
      --max-scan-duration 180 --no-update \
      --format json --output reports/wpscan.json
```

شنو كيدير هاد الأمر: مراجعة إضافات وقوالب WordPress المعروفة ضمن إعداد محافظ.

البارامیطر / الخيار	شنو كيعني وكيدير
--scope / url--	عنوان البداية وقائمة أسماء النطاقات؛ scope ليس تعبير URL منتظماً ولا قيد مسارات.
--enumerate vp,vt	تحديد تعداد الإضافات والقوالب المصنفة ضمن قوائم الثغرات؛ لا تترك e- بلا قيمة.
--detection-mode passive	طريقة الاكتشاف العامة.
plugins-detection / --- themes-detection	طريقة اكتشاف المكونات.
--plugins-version-detection / --themes-version-detection	طريقة استنتاج إصداراتها.
--throttle 1000	فاصل بالميلي ثانية؛ يجعل العمل بخيط واحد.
--request-timeout 10	مهلة الطلب.
--max-scan-duration 180	سقف مدة الفحص بالثواني.
`--no-update`	عدم تحديث قاعدة البيانات ضمن هذه الجولة.
--format json / --output	تقرير JSON محلي.

النتيجة اللي كتسناها: حقول pluginsg version وthemesg إذا اكتشفت؛ وأي scan_aborted عند التوقف.

تحليل النتيجة وكيفاش تقراها: يلزم token لـWPScan API للحصول على بيانات الثغرات؛ إذا لم يكن مضبوطاً فلا تفسر غياب vulnerabilities كسلامة. اضبط WPSCAN_API_TOKEN مسبقاً من حساب WPScan المصرح باستخدامه وفق طريقة حفظ الأسرار المعتمدة. الـAPI خدمة خارجية؛ راجع شروط استخدامها ومشاركة بيانات المكونات. قاعدة قديمة أو كشف إصدار غير واثق قد يعطي نتائج ناقصة.

الأمر 41: تجبد نسخة WordPress ونسبة الثقة فالتحديد

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ jq '{version: .version, scan_aborted: .scan_aborted}' \
      reports/wpscan.json
```

شنو كيدير هاد الأمر: تمييز المعلومة المكتشفة عن فشل الفحص.

البارامیطر / الخيار	شنو كيعني وكيدير
`.version`	تفاصيل إصدار WordPress إن اكتشف.
`.scan_aborted`	سبب إيقاف الفحص إن وجد.
version / scan_aborted	إخراج الحقلين فقط.

النتيجة اللي كتسناها: confidenceg number found_byg بحسب ما أرجعته النسخة، أو null.

تحليل النتيجة وكيفاش تقراها: لا تحول إصداراً مستنتجاً من HTML إلى إثبات CVE. افحص دقة التعرف، نطاق الإصدارات، الإعداد المتأثر ودور المستخدم المطلوب.

الأمر 42: تآفيشي البلاغينات (Plugins) وتنبيهات الثغرات المنطبقة

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ jq '.plugins // {} | to_entries[] | {plugin: .key, version: .value.versio
  n, vulnerabilities: .value.vulnerabilities}' \
      reports/wpscan.json
```

شنو كيدير هاد الأمر: قراءة الإضافات واحدة واحدة.

البارامیطر / الخيار	شنو كيعني وكيدير
plugins. // {}	الإضافات، أو كائن فارغ عند غيابها.
`to_entries[]`	تحويل أسماء الإضافات إلى سجلات.
`.key`	اسم الإضافة.
`.value.version`	إصدارها المكتشف.
`.value.vulnerabilities`	تنبيهات الثغرات إن توفرت بيانات API.

النتيجة اللي كتسناها: سجل لكل إضافة مكتشفة.

تحليل النتيجة وكيفاش تقراها: لكل تنبيه اقرأ المرجع الأصلي ومتطلبات الاستغلال. وجود wp-json أو xmlrpc وحده سلوك طبيعي محتمل. لا تستخدم هذا الفرع لتخمين كلمات مرور أو تعداد مستخدمين.

08 التحقق من صلاحيات API وقراءة ردود شبكات CDN

08 // API ACCESS CONTROL & CDN RESPONSES

من أقوى اختبارات API مقارنة فعل واحد بين هويتين تجريبيتين تملكهما. المثال يفترض أن المورد /api/orders/1001 أنشأه حساب A وأن حساب B لا يملك حق رؤيته. هذا مسار افتراضي لشرح المنهج، وليس ادعاء بوجوده على example.com. في Vercel أو Cloudflare نستمر في اختبار وظيفة التطبيق نفسها؛ لا نستنتج ثغرة من طبقة الاستضافة. مراجع القسم: [30] [31] [32] [33] [3]

الأمر 43: توجد ملفين ديال التوثيق لجوج كونطات تيست (A و B)

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ nano requests/auth-a.conf requests/auth-b.conf
```

شنو كيدير هاد الأمر: حفظ رأسي التوثيق محلياً بدل كتابتهما مباشرة في سطر الأوامر.

البارامیطر / الخيار	شنو كيعني وكيدير
`nano`	تحرير الملفين في جلسة واحدة.
`requests/auth-a.conf`	ملف إعداد رأس التوثيق الخاص بالحساب A.
`requests/auth-b.conf`	ملف إعداد رأس التوثيق الخاص بالحساب B.
header = "Authorization: Bearer demo-token-a"	مثال مكتمل لصيغة محتوى ملف A. القيمة demo-token-a وهمية؛ اكتب رمز حساب الاختبار A الفعلي في الملف المحلي فقط.
header = "Authorization: Bearer demo-token-b"	مثال محتوى ملف B. القيمة demo-token-b وهمية؛ اكتب رمز حساب الاختبار B الفعلي في هذا الملف.

النتيجة اللي كتسناها: ملفان، يحتوي كل واحد السطر المطابق لحسابه فقط.

تحليل النتيجة وكيفاش تقراها: احفظ كل ملف بمحتواه المطابق لحسابه. القيم الموضحة ليست بيانات اعتماد صالحة. لا تضف عناوين URL أو إعداد location أو ملفات تعريف ارتباط إلى ملفي الإعداد. إذا لم تتوفر هويتان مصرح بهما، فلا ينطبق هذا الإجراء. تحتوي الملفات الفعلية أسراراً حتى مع umask 077؛ احتفظ بها محلياً واستبعدها من التقرير.

الأمر 44: طلب المورد بالكونط A (المولى الشرعي للبيانات)

سياق الاستخدام: طلب مصادق عليه

```
$ curl --disable --silent --show-error \
      --config requests/auth-a.conf --max-time 15 \
      --dump-header raw/a-own.headers \
      --output raw/a-own.json \
      'https://example.com/api/orders/1001 '
```

شنو كيدير هاد الأمر: تثبيت مرجع يوضح أن المورد التجريبي موجود ويملكه A.

البارامیطر / الخيار	شنو كيعني وكيدير
`--config`	قراءة رأس التوثيق من ملف A.
--max-time 15	مهلة الطلب.
--dump-header / --output	حفظ الرؤوس والجسم.
`/api/orders/1001`	معرف مورد تجريبي معروف مسبقاً؛ ليس قيمة يجري تخمينها.

النتيجة اللي كتسناها: الرد المتفق عليه في API، غالباً 200 وJSON يخص A.

تحليل النتيجة وكيفاش تقراها: راجع هوية المالك من السجل الذي أنشأته. إذا كان المورد عاماً أوغير موجود، فلن تكون مقارنة حساب B اختبار IDOR صالحاً.

الأمر 45: تجرب تطلب نفس المورد ديال A بالكونط B (تيست IDOR)

سياق الاستخدام: تحقق صلاحيات مصرح

```
$ curl --disable --silent --show-error \
      --config requests/auth-b.conf --max-time 15 \
      --dump-header raw/b-other.headers \
      --output raw/b-other.json \
      'https://example.com/api/orders/1001 '
```

شنو كيدير هاد الأمر: تغيير الهوية فقط، مع إبقاء المورد التجريبي نفسه.

البارامیطر / الخيار	شنو كيعني وكيدير
`auth-b.conf`	هوية B التي لا يفترض أن تملك المورد.
b-other.headers / b-other.json	أسماء منفصلة لأدلة المحاولة.
باقي الخيارات	نفس طريقة المرجع حتى لا نغير متغيرين في التجربة.

النتيجة اللي كتسناها: رفض مصمم مثل 403 أو 404، أو الاستجابة التي يحددها عقد التطبيق.

تحليل النتيجة وكيفاش تقراها: إذا أعاد بيانات A الخاصة فعلاً فقد يكون BOLA أو IDOR. 200 وحدها لا تكفي؛ قد تكون رسالة خطأ JSON أو صفحة دخول. توقف عند المورد الاصطناعي الذي تملكه؛ لا تعدد معرفات أشخاص آخرين.

الأمر 46: مقارنة الحقول والبيانات اللي رجعو فالريسبونسين بجوج

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ jq -c '{id, ownerId, status, error}' \
      raw/a-own.json raw/b-other.json
```

شنو كيدير هاد الأمر: عرض حقول قليلة بدلاً من نسخ كامل البيانات.

البارامیطر / الخيار	شنو كيعني وكيدير
`-c`	كائن JSON مضغوط في سطر.
{id, ownerId, status, error}	اختيار الحقول إن كانت هذه أسماءها في API.
الملفان	يقرأهما وز بالترتيب؛ السطر الأول لـA والثاني لـB.

النتيجة اللي كتسناها: سطران يمكن مقارنتهما.

تحليل النتيجة وكيفاش تقراها: عدّل الحقول وفق بنية API الفعلية؛ null قد يعني اسم حقل مختلفاً. الدليل القوي: B يحصل على بيانات خاصة بمورد A رغم عدم استحقاقه وفق تصميم التطبيق، مع طلبين قابلين للتكرار ودون عرض أسرار زائدة.



الشكل 3: مصفوفة التحقق من عزل الصلاحيات ومنطق التحكم فالوصول (IDOR / BOLA)

الأمر 47: قراءة ترويسات الكاش (Cache Headers) والتحكم فيها

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ grep -niE \
      'cache-control:|vary:|age:|x-vercel-cache:|cf-cache-status:' \
      raw/a-own.headers raw/b-other.headers
```

شنو كيدير هاد الأمر: فهم هل يظهر اختلاف متعلق بالكاش بين طلبي الحسابين.

الباراميطر / الخيار	شنو كيعني وكيدير
grep -niE	بحث في ملفين مع أسماء الملفات وأرقام السطور.
cache-control / vary	سياسة التخزين وعوامل تنويع الرد.
age / x-vercel-cache / cf-cache-status	قرائن على عمر الرد أوطبقة الكاش.

النتيجة اللي كتسناها: رؤوس مطابقة إذا أرسلها الخادم أو CDN.

تحليل النتيجة وكيفاش تقراها: HIT لا يثبت تسرب بيانات، وMISS لا يثبت غياب كل أنواع الكاش. إثبات التسرب يحتاج وصول حساب B لبيانات A الخاصة. لا تبدأ cache poisoning أو تغيير Host أو البحث عن origin لتجاوز WAF ضمن هذه المراجعة.

▪ كيفاش تكمل الفحص على حساب التيكنولوجي اللي لقيتي

إلا كان الموقع WordPress، كتمشي نيشان لـ WPScan وفحص إضافات وقوالب الموقع. إلا كان API، كتركز على عقد الطلبات (Schemas) وعزل الملكية (IDOR/BOLA). إلا كان تطبيق React/Next.js على Vercel، كتركز على المنطق ديال الـ API ومسارات السيرفر. ملي كيبان ليك cf-mitigated: challenge أو الكود 429، كتعرف أن طبقة الحماية أو WAF كدير خدمتها، وما تضيعش وقتك فمحاولة فك تشفير البروكسي المشترك.

09 كشف الثغرات فـ لّاب محلي (Local Lab)

09 // LOCAL LAB EXPLOITATION (SQLMAP & ZAP)

هذا الجزء يستعمل نسخة OWASP Juice Shop مجهزة مسبقاً على جهاز المختبر في http://127.0.0.1:3000 ببيانات اصطناعية قابلة للإرجاع. لا يتضمن تثبيت المختبر. 127.0.0.1 يعني الجهاز أو البيئة التي تنفذ الأمر فيها؛ إذا كان المختبر في VM أخرى فلن يشير إليها. تحقق من هوية الخدمة وعزل بيئة الاختبار، ومن عدم توجيه الطلبات إلى أنظمة خارجية بواسطة التحويلات أو إعدادات الوكيل. مراجع القسم: [34] [35] [37] [38] [39] [36] [40]

الأمر 48: تفحص مسار البحث فاللّاب المحلي قبل تيست الـ SQLi

سياق الاستخدام: طلب مختبر محلي

```
$ curl --disable --silent --show-error --noproxy '*' \
      --max-time 10 --dump-header raw/lab.headers \
      --output raw/lab-search.json \
      'http://127.0.0.1:3000/rest/products/search?q=apple '
```

شنو كيدير هاد الأمر: التأكد أن مختبر Juice Shop الصحيح يرد على مسار البحث.

الباراميطر / الخيار	شنو كيعني وكيدير
--noproxy '*'	تجاوز البروكسي لهذا الأمر حتى يبقى الاتصال محلياً.
`127.0.0.1:3000`	خدمة المختبر المحلية؛ ليست example.com.
`/rest/products/search`	مسار البحث في Juice Shop.
q=apple	قيمة بحث مرجعية عادية.
--dump-header / --output	حفظ دليل الأساس.

النتيجة اللي كتسناها: JSON لنتيجة البحث إذا كان المختبر قيد التشغيل ومتوافقاً.

تحليل النتيجة وكيفاش تقراها: Connection refused يعني غياب الخدمة أو خطأ المنفذ؛ لا تبدل العنوان إلى موقع عشوائي. تأكد أن الجسم JSON المتوقع وليس موقعاً مختلفاً أو صفحة دخول.

الأمر 49: تيست مقيد لثغرة SQL injection بـ sqlmap فالمختبر المحلي

سياق الاستخدام: فحص هجومي في المختبر فقط

```
$ timeout 180s sqlmap \
      -u 'http://127.0.0.1:3000/rest/products/search?q=apple' \
      -p q --batch --level=1 --risk=1 --technique=B \
      --threads=1 --delay=1 --timeout=10 --retries=0 \
      --ignore-proxy --ignore-redirects \
      --output-dir=reports/sqlmap-lab
```

شنو كيدير هاد الأمر: رؤية كيف تختبر أداة متخصصة تأثير إدخال المستخدم في استعلام SQL.

البارامیطر / الخيار	شنو كيعني وكيدير
timeout 180s	إنهاء الأداة بعد ثلاث دقائق؛ قد يقطع الفحص قبل اكتماله.
sqlmap -u	الأداة ورابط المختبر.
-p q	اختبار المعامل q فقط.
`--batch`	إجابات افتراضية دون أسئلة تفاعلية؛ ليس ضمان أمان.
--level=1 / --risk=1	أقل مستويين للاختبارات المتاحة، مع بقاء الفحص نشطاً.
--technique=B	اختبارات boolean based فقط.
--threads=1 / --delay=1	خيط واحد وتأخير ثانية بين الطلبات.
--timeout=10 / --retries=0	مهلة الطلب دون إعادة.
`--ignore-proxy`	تجاهل proxy النظام.
`--ignore-redirects`	عدم متابعة إعادة التوجيه من المختبر.
`--output-dir`	حفظ سجل الجلسة والأدلة محلياً.

النتيجة اللي كتسناها: قد يظهر أن q injectable، أو عدم اكتشاف، أوانتهاء المهلة.

تحليل النتيجة وكيفاش تقراها: راجع payload والطلب الفعلي وأعد مقارنة شرط true وfalse على بيانات المختبر. لا يوجد في هذا المثال dump أو os-shell. مستوى risk منخفض لا يجعل تشغيل الأمر على إنتاج مباحاً أوخالياً من الأثر. حد الثلاث دقائق قد يعطي فحصاً ناقصاً ولا يثبت السلامة.

الأمر 50: قراءة الدلائل واللوغز اللي سجلاتهم sqlmap

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ grep -RniE 'Parameter:|Type:|Title:|Payload:' \
      reports/sqlmap-lab --include=log
```

شنو كيدير هاد الأمر: استخراج وصف نقطة الحقن وتقنيتها من ملفات السجل.

البارامیطر / الخيار	شنو كيعني وكيدير
`-R`	البحث في المجلدات الفرعية.
n / -i / -E-	أرقام سطور وحالة حروف مرنة ونمط موسع.
include=log--	حصر القراءة في الملفات التي اسمها log.
Parameter / Type / Title / Payload	المعامل والتقنية والعنوان والقيمة التي سجلتها الأداة.

النتيجة اللي كتسناها: السطور الموجودة فقط؛ قد لا يوجد log نهائي إذا قطعت العملية مبكراً.

تحليل النتيجة وكيفاش تقراها: payload في السجل جزء من سجل الاختبار المحلي، وليس أمراً يعاد على نطاق حقيقي. نجاح أداة لا يغني عن مقارنة مرجع وتحقق مستقل؛ لا تنتقل إلى استخراج قواعد البيانات لمجرد إثبات الاسم.

الأمر 51: تفتح OWASP ZAP باش تراقب وتفلي الترافيك

سياق الاستخدام: واجهة محلية

```
$ zaproxy
```

شنو كيدير هاد الأمر: تشغيل واجهة ZAP المثبتة في Kali.

البارامیطر / الخيار	شنو كيعني وكيدير
`zaproxy`	مشغل التطبيق.

النتيجة اللي كتسناها: نافذة ZAP.

تحليل النتيجة وكيفاش تقراها: اختر Safe Mode للمراجعة السلبية، واستعمل بروكسي loopback ومنفذاً مختلفاً عن Burp عند تشغيلهما معاً. Protected Mode يسمح بأفعال خطرة على ما عرّفته in scope، لذلك دقة context مهمة. Passive Scan يحلل المرور؛ زيارة الصفحات نفسها ما زالت ترسل طلبات.

الأمر 52: فحص أوتوماتيكي بـ ZAP فالمختبر المحلي فقط

سياق الاستخدام: فحص هجومي في المختبر فقط

```
$ timeout 300s zaproxy -cmd \
      -quickurl http://127.0.0.1:3000 \
      -quickout "$PWD/reports/zap-lab.json" -quickprogress
```

شنو كيدير هاد الأمر: تجربة زحف وفحص نشط داخل مختبر قابل للإرجاع فقط.

الباراميطر / الخيار	شنو كيعني وكيدير
timeout 300s	خمس دقائق كحد للجولة.
`-cmd`	تشغيل من سطر الأوامر.
`-quickurl`	عنوان سيزحف إليه ZAP ويفحصه بنشاط.
`-quickout`	حفظ التقرير بصيغة تحددها اللاحقة json.
`$PWD`	المسار الحالي الكامل.
`-quickprogress`	عرض التقدم النصي.

النتيجة اللي كتسناها: تقرير إن اكتمل الفحص؛ قد لا يكتب عند قطع المهلة.

تحليل النتيجة وكيفاش تقراها: أغلق نسخة ZAP الأخرى أو خصص مجلد بيانات مختلفاً قبل الأمر لتجنب قفل الجلسة. الفحص النشط يرسل payloads وقد يغير بيانات المختبر، Quick Start لا يغطي كل مسارات تطبيق JavaScript. لا تستبدل localhost بـexample.com؛ على الأهداف المصرح بها راجع سياق الفحص والطلبات والمعدل من الواجهة أولاً.

الأمر 53: قراءة وتفريك تنبيهات ZAP من فيشي JSON

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ jq -r '.site[]?.alerts[]? | [.alert, .riskdesc, .confidence, .count] | @tsv' reports/zap-lab.json
```

شنو كيدير هاد الأمر: عرض التنبيهات ودرجة الثقة وعدد مواضعها.

الباراميطر / الخيار	شنو كيعني وكيدير
.site[]?.alerts[]?	التنبيهات في المواقع المسجلة إن وجدت.
`.alert`	الاسم.
`.riskdesc`	وصف الخطورة.
`.confidence`	درجة الثقة كما يسجلها ZAP.
`.count`	عدد الحالات المسجلة.
@tsv	أعمدة نصية.

النتيجة اللي كتسناها: صف لكل تنبيه في تقرير ZAP التقليدي.

تحليل النتيجة وكيفاش تقراها: إذا كانت بنية تقرير نسختك مختلفة، اقرأ JSON أولاً ولا تفترض أن غياب الأسطر يعني صفر تنبيهات. راجع URIs وinstances وparameterg evidenceg ثم اختبر أقل حالة في المختبر.

10 تحليل الأوامر مرتفعة الخطورة (أمثلة معطلة ومخاطرها)

10 // HIGH-RISK & DISABLED COMMANDS ANALYSIS

تعرض الأمثلة التالية قيماً واضحة بدلاً من عناصر نائبة غير مكتملة. جميع الأسطر معلّقة بعلامة # لقراءتها وتحليلها فقط. يشير example.com إلى نطاق توثيقي حقيقي تديره IANA؛ أما 127.0.0.1 فيشير إلى الجهاز أو البيئة التي يعمل فيها الأمر. استخدام عنوان محلي لا يجعل الإغراق أو الحذف آمناً، لذلك تبقى هذه الأمثلة معطّلة. تتحدد المخاطر بالفعل والخيارات والهدف وحجم الطلبات والصلاحيات، وليس باسم الأداة وحده. مراجع القسم: [35] [19] [22] [21] [3] [4] [41] [42] [5]

الأمر 54: التيليشارجي سكريبت وترانيه نيشان بـ sudo (خطر كبير!)

سياق الاستخدام: تحليل مخاطر فقط — أمر معطّل وما خصكش ترانيه!

```
# # curl https://example.com/script.sh | sudo bash
```

شنو كيدير هاد الأمر: توضيح مسار انتقال المحتوى من عنوان ويب إلى مفسر أوامر يعمل بصلاحيات مرتفعة.

البارامیطر / الخيار	شنو كيعني وكيدير
`#`	تعليق السطر ومنع تنفيذه في Bash.
`curl`	جلب محتوى العنوان وكتابته إلى الخرج القياسي.
`https://example.com/script.sh`	رابط توضيحي كامل، دون أقواس. لا يفترض المثال أن هذا الملف موجود أو أنه برنامج موثوق.
`|`	تمرير خرج curl إلى الإدخال القياسي للأمر التالي.
`sudo`	طلب صلاحيات مرتفعة للأمر الذي يليه على الجهاز المحلي.
`bash`	قراءة النص الوارد بوصفه أوامر وتنفيذ ما تسمح به صيغته وصلاحيات العملية.

النتيجة اللي كتسناها: لا يحدث تنفيذ: كل سطر يبدأ بعلامة #، وهي علامة تعليق في Bash.

تحليل النتيجة وكيفاش تقراها: عند إزالة # قد يُنفّذ المحتوى المنزّل على جهاز التشغيل بصلاحيات مرتفعة إذا كان شيفرة صالحة. لا يفحص هذا السطر الموقع بالضرورة؛ مصدر الخطر هو تنفيذ محتوى بعيد دون مراجعته. قد يكون الرد صفحة HTML أو رسالة خطأ، لكن فشل هذه الاستجابة لا يجعل الأسلوب آمناً. تقتضي مراجعة أي سكربت التحقق من مصدره ومحتواه وسلامته قبل منحه صلاحيات.

الأمر 55: إرسال باكيطات بلا تحديد المعدل (DDoS / إغراق خايب)

سياق الاستخدام: تحليل مخاطر فقط — أمر معطّل وما خصكش ترانيه!

```
# # sudo hping3 -S -p 8000 --flood 127.0.0.1
```

شنو كيدير هاد الأمر: تحديد الخيار الذي يحوّل توليد الحزم إلى إرسال بأقصى سرعة ممكنة.

الباراميطر / الخيار	شنو كيعني وكيدير
`#`	يبقي المثال معطّلاً.
`sudo`	صلاحيات محلية قد تتطلبها المقابس الخام.
`hping3`	أداة لإنشاء حزم شبكة مخصصة.
`-S`	ضبط علامة SYN في حزم TCP.
p 8000-	تحديد منفذ الوجهة 8000.
`--flood`	الإرسال بأقصى سرعة ممكنة دون عرض الردود؛ ليس اختباراً محدود الطلبات.
`127.0.0.1`	واجهة loopback للبيئة الحالية؛ ليست عنوان موقع خارجي.

النتيجة اللي كتسناها: لا يحدث تنفيذ: كل سطر يبدأ بعلامة #، وهي علامة تعليق في Bash.

تحليل النتيجة وكيفاش تقراها: قد يستهلك هذا النمط موارد الشبكة أو النظام ويؤثر في توافر الخدمة، حتى داخل الجهاز المحلي. لا تثبت استجابة الخادم للإغراق وجود ثغرة مقبولة في برنامج Bug Bounty. أي اختبار سعة يحتاج خطة مستقلة وحدوداً قابلة للقياس وآلية إيقاف؛ هذا السطر ليس تلك الخطة.

الأمر 56: محاولات لوغين لا نهائية بقواميس ضخمة (Brute-force عشوائي)

سياق الاستخدام: تحليل مخاطر فقط — أمر معطّل وما خصكش ترانيه!

```
# # hydra -L requests/users.txt -P requests/passwords.txt 127.0.0.1 ssh
```

شنو كيدير هاد الأمر: التعرف إلى تركيب أمر تخمين بيانات اعتماد من ملفين.

البارامیطر / الخيار	شنو كيعني وكيدير
`#`	تعطيل المثال.
`hydra`	أداة لاختبار محاولات المصادقة المتكررة.
L requests/users.txt-	قراءة أسماء الحسابات من ملف محلي، اسم واحد في كل سطر.
P requests/passwords.txt-	قراءة كلمات المرور من ملف محلي، قيمة واحدة في كل سطر.
`127.0.0.1`	الخدمة المحلية الافتراضية في المثال.
`ssh`	اختيار بروتوكول SSH؛ ليس طلب HTTP إلى صفحة تسجيل دخول.

النتيجة اللي كتسناها: لا يحدث تنفيذ: كل سطر يبدأ بعلامة #، وهي علامة تعليق في Bash.

تحليل النتيجة وكيفاش تقراها: قد تتسبب المحاولات المتكررة في قفل الحسابات أو الضغط على الخدمة أو الدخول غير المصرح. وجود أداة الاختبار أو ملفات القوائم لا يمنح إذناً لاستخدامها. أسماء الملفين توضيحية؛ لا ينشئهما هذا السطر ولا يفترض أنهما موجودان.

الأمر 57: الانتقال من كشف ثغرة SQLi لاستخراج وتنزيل الداتا كاملة (ممنوع!)

سياق الاستخدام: تحليل مخاطر فقط — أمر معطّل وما خصكش ترانيه!

```
# # sqlmap -u 'http://127.0.0.1:3000/rest/products/search?q=apple' \
      # --dump-all
```

شنو كيدير هاد الأمر: تمييز التحقق من وجود SQL injection عن طلب استخراج محتويات قواعد البيانات.

البارامیطر / الخيار	شنو كيعني وكيدير
`#`	كل سطر معطّل على حدة.
`sqlmap`	أداة يمكنها الكشف أو استخراج البيانات بحسب خياراتها والشروط المتاحة.
`-u`	تحديد رابط الطلب الذي ستختبره الأداة.
http://127.0.0.1:3000/rest/products/search?q=apple	مسار بحث توضيحي داخل المختبر المحلي، مع قيمة q محددة.
`--dump-all`	طلب استخراج بيانات جميع قواعد البيانات التي تستطيع الأداة الوصول إليها.

النتيجة اللي كتسناها: لا يحدث تنفيذ: كل سطر يبدأ بعلامة #، وهي علامة تعليق في Bash.

تحليل النتيجة وكيفاش تقراها: هذا الخيار يوسّع الأثر إلى قراءة بيانات قد تتضمن أسراراً ومعلومات شخصية، وقد يطيل الفحص ويزيد الطلبات. النتيجة مشروطة بوجود حقن قابل للاستغلال وصلاحيات مناسبة؛ ليس الخيار ضماناً لاستخراج كل شيء. لا يلزم جمع بيانات حقيقية لإثبات نقطة حقن على بيانات اختبار مملوكة.

الأمر 58: طلب تنفيذ أوامر طرفية على سيستيم السيرفور (RCE غير مسموح)

سياق الاستخدام: تحليل مخاطر فقط — أمر معطّل وما خصكش ترانيه!

```
# # sqlmap -u 'http://127.0.0.1:3000/rest/products/search?q=apple' \
      # --os-shell
```

شنو كيدير هاد الأمر: تحديد خيار يحاول نقل الاستغلال من طبقة قاعدة البيانات إلى تنفيذ أوامر النظام.

البارامیطر / الخيار	شنو كيعني وكيدير
`#`	يبقي كلا السطرين معطّلين.
sqlmap -u	الأداة ورابط المختبر المحلي المحدد.
`--os-shell`	محاولة توفير جلسة أوامر نظام عند تحقق شروط قاعدة البيانات والامتيازات والبيئة.

النتيجة اللي كتسناها: لا يحدث تنفيذ: كل سطر يبدأ بعلامة #، وهي علامة تعليق في Bash.

تحليل النتيجة وكيفاش تقراها: هذا انتقال إلى قدرة قد تغيّر النظام أو ملفاته، وقد يتضمن إنشاء مكونات وسيطة على الخادم. اكتشاف معامل قابل للحقن لا يثبت توفر تنفيذ أوامر، ولا يبيح اختبار هذه القدرة. وجود المثال هنا يوضح الفرق بين مستويات الأثر؛ لا يتضمن إجراء استغلال فعلي.

الأمر 59: توسيع سكان Nikto بفئات حساسة ومزعجة للخادم

سياق الاستخدام: تحليل مخاطر فقط — أمر معطّل وما خصكش ترانيه!

```
# # nikto -h http://127.0.0.1:3000 -Tuning 68
```

شنو كيدير هاد الأمر: بيان أن إعداد فئات الفحص يغيّر طبيعة الاختبارات التي تجريها الأداة نفسها.

الباراميطر / الخيار	شنو كيعني وكيدير
`#`	تعطيل السطر.
`nikto`	ماسح ويب يعتمد على اختبارات وإضافات.
h http://127.0.0.1:3000-	عنوان المختبر المحلي.
Tuning 68-	اختيار الفئتين 6 و8 معاً؛ القيمة ليست عدد الطلبات.
`6`	فئة اختبارات حجب الخدمة وفق تصنيف Nikto.
`8`	فئة تنفيذ الأوامر أو الصدفة البعيدة وفق تصنيف الأداة.

النتيجة اللي كتسناها: لا يحدث تنفيذ: كل سطر يبدأ بعلامة #، وهي علامة تعليق في Bash.

تحليل النتيجة وكيفاش تقراها: لا تجعل معرفة اسم الماسح إعداداته جميعها ملائمة للهدف. قد تتضمن هذه الفئات طلبات ذات أثر يتجاوز المراجعة المحدودة، وتختلف الاختبارات المتاحة باختلاف قاعدة الأداة وإعداداتها. راجع الاختبار نفسه وشروط البرنامج؛ اسم الفئة لا يضمن نجاح الاستغلال أو حدوث أثر محدد.

الأمر 60: سكان واسع لآلاف البورتات بـ Nmap وبسرعة خيالية

سياق الاستخدام: تحليل مخاطر فقط — أمر معطّل وما خصكش ترانيه!

```
# # nmap -sT -p- --min-rate 1000 127.0.0.1
```

شنو كيدير هاد الأمر: توضيح الفرق بين وضع سقف للمعدل ومحاولة فرض حد أدنى مرتفع.

البارامیطر / الخيار	شنو كيعني وكيدير
`#`	تعطيل المثال.
`nmap`	ماسح منافذ وخدمات شبكة.
`-sT`	فحص اتصالات TCP باستخدام واجهة اتصال نظام التشغيل.
`-p-`	فحص منافذ TCP من 1 إلى 65535 في هذا المثال.
--min-rate 1000	طلب حد أدنى قدره 1000 حزمة في الثانية عندما تسمح الأداة والبيئة؛ ليس سقفاً.
`127.0.0.1`	عنوان الجهاز المحلي في المثال.

النتيجة اللي كتسناها: لا يحدث تنفيذ: كل سطر يبدأ بعلامة #، وهي علامة تعليق في Bash.

تحليل النتيجة وكيفاش تقراها: قد يزيد اتساع المنافذ والمعدل المرتفع الضغط ويقلل موثوقية النتائج. المعدل الفعلي يتأثر بالشبكة ونوع الفحص؛ القيمة ليست ضماناً لعدد ثابت من الطلبات. الخيار --max-rate يضع سقفاً، بينما --min-rate يطلب حداً أدنى؛ لا يتبادلان المعنى.

الأمر 61: خدمة قوالب Nuclei فيها كود أو سكربتات بلا مراجعة

سياق الاستخدام: تحليل مخاطر فقط — أمر معطّل وما خصكش ترانيه!

```
# # nuclei -u http://127.0.0.1:3000 \
      # -t templates/unreviewed-code.yaml -code
```

شنو كيدير هاد الأمر: تحديد إعداد يسمح بتشغيل قوالب بروتوكول code.

البارامیطر / الخيار	شنو كيعني وكيدير
`#`	تعطيل كل سطر من المثال.
u http://127.0.0.1:3000-	عنوان المختبر المحلي.
t templates/unreviewed--code.yaml	اسم ملف قالب توضيحي غير مقدم في الدليل؛ الاسم وحده لا يصف محتواه أو موثوقيته.
`-code`	تمكين قوالب بروتوكول code؛ تبقى متطلبات الثقة والتوقيع الخاصة بالإصدار سارية.

النتيجة اللي كتسناها: لا يحدث تنفيذ: كل سطر يبدأ بعلامة #، وهي علامة تعليق في Bash.

تحليل النتيجة وكيفاش تقراها: يمكن لشيفرة القالب أن تعمل على جهاز تشغيل Nuclei، إلى جانب الطلبات التي ترسلها إلى الهدف. لا يكفي عنوان القالب أو تصنيف severity أو وجود توقيع للحكم على ملاءمته. اقرأ الشيفرة والملفات والاتصالات التي يستخدمها قبل اعتماد أي قالب.

الأمر 62: مسح دواسى محليين بـ rm -rf مع صلاحيات sudo (خطر مسح السيستيم)

سياق الاستخدام: تحليل مخاطر فقط — أمر معطّل وما خصكش ترانيه!

```
# # sudo rm -rf ./lab-disposable-files
```

شنو كيدير هاد الأمر: توضيح أن بعض الأوامر الخطرة تؤثر في ملفات الجهاز المحلي، لا في الموقع.

البارامیطر / الخيار	شنو كيعني وكيدير
`#`	يبقي المثال معطّلاً.
`sudo`	تشغيل الحذف بصلاحيات مرتفعة إذا سمحت السياسة المحلية.
`rm`	حذف ملفات أو مجلدات.
`-r`	الحذف المتكرر لمحتويات المجلدات.
`-f`	عدم طلب التأكيد المعتاد وتجاهل الملفات غير الموجودة.
`./lab-disposable-files`	مجلد اسمه lab-disposable-files داخل المجلد الحالي؛ قيمة توضيحية مكتملة.

النتيجة اللي كتسناها: لا يحدث تنفيذ: كل سطر يبدأ بعلامة #، وهي علامة تعليق في Bash.

تحليل النتيجة وكيفاش تقراها: يتحدد موقع الحذف بالمجلد الحالي، وقد تؤدي الصلاحيات المرتفعة إلى إزالة ملفات لا يستطيع الحساب حذفها عادة. حذف مجلد محلي يحمل اسم موقع لا يحذف الموقع نفسه. لا تنفذ حذفاً متكرراً قبل التحقق من المسار المطلق والمحتويات ووجود نسخة قابلة للاسترجاع.

- **القاعدة الذهبية باش تقرا أي أمر قبل ما تلونصيه فالتيرمينال**

قبل ما تضغط Enter على أي أمر كتلقاه فـ GitHub ولا التوتوريالات: حدد أولاً فين غينزل الأثر (فالبيئة ديالك المحلية ولا فالسيرفور البعيد؟). من بعد، حدد الفعل: واش غير كيقرا، ولا كيكرر الطلبات بمعدل عالي، ولا كيجرب استخراج داتا، ولا كينفذ كود، ولا كيمسح فيشيات! راجع عدد الطلبات والتوازي (Concurrency) والوقت المسموح والصلاحيات. افصل ديما بين علامة التعليق # فالدليل وبين الأمر القابل للتنفيذ؛ وما تزيدش sudo إلا إلا كان الأمر كيتطلبها محلياً وبشكل مبرر.

11 من النتيجة ديال الأداة لتقرير بروفيسيونيل وموثق

11 // REPRODUCIBLE REPORTING & HASHING

اكتب الملاحظة كما رأيتها، ثم الفرضية، ثم أثر التحقق المحدود. المسار الموجود والإصدار القديم والتطابق الآلي ورفض WAF ليست درجات خطورة جاهزة. التقرير الجيد يربط طلباً مسموحاً بسلوك غير متوقع ودليل ونتيجة يمكن تكرارها. مراجع القسم: [30] [2]

الأمر 63: توثيق النتيجة وتحديد الأثر ونسبة الثقة التقنية

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ nano notes/finding.md
```

شنو كيدير هاد الأمر: إنشاء تقرير قصير مبني على الدليل.

البارامیطر / الخيار	شنو كيعني وكيدير
`nano`	تحرير.
`notes/finding.md`	مسودة التقرير المحلية.

النتيجة اللي كتسناها: ملف تكتب فيه الهدف والطريقة والهوية والخطوات والنتيجة.

تحليل النتيجة وكيفاش تقراها: اكتب: النطاق؛ وقت الاختبار؛ الشرط المسبق؛ المتوقع؛ الفعلي؛ الدليل؛ الأثر؛ ما بقي غير مؤكد؛ معالجة مقترحة. احذف tokens وcookies والبيانات الزائدة من نسخة المشاركة. لا تسمّ البلاغ Critical اعتماداً على تسمية الأداة.

الأمر 64: حساب بصمة الهاش (SHA-256) لكاع فيشيات الأدلة والتقارير

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ sha256sum raw/root.headers raw/root.html \
      > reports/evidence.sha256
```

شنو كيدير هاد الأمر: تسجيل بصمات الملفات الأساسية حتى نلاحظ أي تعديل لاحق.

البارامیطر / الخيار	شنو كيعني وكيدير
`sha256sum`	حساب SHA-256.
ملفا raw	المحتوى الذي نحسب بصمته.
> reports/evidence.sha256	حفظ البصمات محلياً.

النتيجة اللي كتسناها: سطر ببصمة لكل ملف.

تحليل النتيجة وكيفاش تقراها: البصمة تكشف تغير المحتوى إذا قارنتها بمرجع موثوق؛ لا تثبت أن المصدر صحيح أو أن المحتوى ثغرة. أضف ملفات التحقق ذات الصلة دون أرشفة أسرار التوثيق.

15 المراجع والدوكيمونطاسيون الرسمية

OFFICIAL DOCUMENTATION REFERENCES

أرقام المراجع فبداية كل قسم كتشير للوثائق والمصادر الرسمية المرتبطة به. راجع ديما الدوكيمونطاسيون ديال الإصدار اللي مأنستالي عندك إلا شفتي شي خيار تبدل. بعض المراجع فيها التثبيت ولكن هاد الدليل مخصص للاستخدام والتطبيق المباشر.

- [1] IANA نطاقات الأمثلة
- [2] HackerOne قواعد السلوك والنطاق
- [3] curl مرجع الخيارات
- [4] GNU Bash مرجع الصدفة والتعليقات والتوجيه
- [5] GNU Coreutils مرجع أدوات الملفات
- [6] Kali WhatWeb والمدخلات المحلية
- [7] Kali تشغيل Burp Suite
- [8] PortSwigger العمل داخل Repeater
- [9] PortSwigger إعدادات Repeater والتحويلات
- [10] PortSwigger الفحص ضمن الاختبار اليدوي
- [11] PortSwigger إدارة شهادة Burp
- [12] Postman مشغل Linux والتمييز عن CLI
- [13] Postman CLI تشغيل collections وخياراتها
- [14] Postman صيغ التقارير وبنية Newman
- [15] Newman خيارات تشغيل المجموعات
- [16] Newman التقارير المدمجة
- [17] Postman كتابة assertions
- [18] ffuf مرجع الاستخدام والخيارات
- [19] Kali Nikto وفئات الفحص
- [20] Nmap أساليب فحص المنافذ
- [21] Nmap حدود المعدل والتوقيت
- [22] Nuclei مرجع خيارات التشغيل
- [23] Nuclei بنية طلب HTTP
- [24] Nuclei شروط المطابقة
- [25] Kali خيارات WPScan
- [26] WPScan إعداد API Token
- [27] WPScan خيارات تعداد القوالب والإضافات
- [28] WordPress اكتشاف REST API
- [29] CMSScanner معنى scope وحد مدة الفحص
- [30] OWASP WSTG اختبار IDOR
- [31] Vercel رؤوس الاستجابة
- [32] Cloudflare اكتشاف صفحة التحدي
- [33] Cloudflare معنى proxy status
- [34] OWASP Juice Shop شيفرة البحث في المختبر
- [35] sqlmap خيارات الكشف ومستويات الاختبار
- [36] Kali مشغل ZAP
- [37] ZAP بداية الاختبار والفرق بين الفحصين
- [38] ZAP Quick Start وخيارات سطر الأوامر
- [39] ZAP أوضاع التشغيل
- [40] ZAP تقرير JSON التقليدي

[41] Hydra مرجع الأداة

[42] Kali hping3 وخيار flood

16 مثال تطبيقي متسلسل خطوة بخطوة على نطاق تجريبي (www.example.com)

END-TO-END WORKFLOW
(WWW.EXAMPLE.COM)

العنوان في هذا المثال هو https://www.example.com، وهو اسم مضيف يختلف عن example.com عند تحديد نطاق الاختبار. يوضح التسلسل إنشاء مجلد جلسة، وحفظ استجابة، وتحليلها، ثم استخدام قائمة مسارات صغيرة وقالب Nuclei محدد، وتوثيق النتيجة. تبقى هذه صياغة توضيحية؛ استبدل الهدف ببيئة مصرح بها قبل تشغيل أوامر الاتصال. المجلد session-01 جلسة مستقلة. إذا كان موجوداً وفيه أدلة سابقة، اختر اسماً جديداً وعدّل أمري إنشاء المجلد والدخول إليه معاً. لا يفترض المثال أن API أو WordPress أو ملف Git موجود على نطاق الأمثلة، ولا أن أي استجابة متوقعة أدناه قد تحققت فعلياً.

خطوة المثال 01: صاوب الدوسي ديال التارغت (Target)

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ mkdir -p ~/bug-bounty/www.example.com/session-01
```

شنو كيدير هاد الأمر: كتصاوب بلاصة مجمّعة للملاحظات والريكويستات والتقارير.

البارامیطر / الخيار	شنو كيعني وكيدير
`mkdir`	إنشاء مجلد.
`-p`	إنشاء المجلدات الأب عند الحاجة؛ لا يفشل إذا كان المجلد موجوداً.
`~`	مجلد المستخدم الحالي.
`bug-bounty/www.example.com/session-01`	مسار محلي لتنظيم ملفات الهدف؛ لا يرسل طلباً للموقع.

النتيجة اللي كتسناها: فالعادة ما كيطلع حتى تيكست فالتيرمينال إلا داز الأمر بنجاح.

تحليل النتيجة وكيفاش تقراها: Permission denied كتعني عندك مشكل فصلاحيات الكتابة فالسيستيم. بدّل المسار لبلاصة كتحكم فيها، وما تحلش المشكل بأنك تزيد sudo على أي حاجة! اختر اسم جلسة جديداً عند التكرار للحفاظ على الأدلة السابقة.

خطوة المثال 02: دخل للدوسي ديال الخدمة

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ cd ~/bug-bounty/www.example.com/session-01
```

شنو كيدير هاد الأمر: جعل الملفات التالية داخل مجلد الهدف.

البارامیطر / الخيار	شنو كيعني وكيدير
`cd`	تغيير المجلد الحالي.
~/bug-bounty/www.example.com/session-01	المجلد الذي أنشأناه.

النتيجة اللي كتسناها: يتغير المسار في موجه Terminal غالباً.

تحليل النتيجة وكيفاش تقراها: إذا ظهر No such file or directory، أصلح المسار قبل المتابعة.

خطوة المثال 03: تأكد من البلاصة فين راك كاين فالتيرمينال (pwd)

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ pwd
```

شنو كيدير هاد الأمر: إظهار المسار الكامل.

البارامیطر / الخيار	شنو كيعني وكيدير
`pwd`	اختصار print working directory؛ اطبع مجلد العمل.

النتيجة اللي كتسناها: /home/kali/bug-bounty/www.example.com/session-01

تحليل النتيجة وكيفاش تقراها: المسار المعروض يفترض أن اسم المستخدم المحلي هو kali. يتغير هذا الجزء بحسب حسابك، بينما يظل مجلد الهدف www.example.com كما أنشأته. تحقق من المجلد الحالي قبل حفظ الأدلة.

خطوة المثال 04: قفل الصلاحيات باش ما يشوف حتى واحد الفيشيات الجداد (umask)

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ umask 077
```

شنو كيدير هاد الأمر: تقليل وصول مستخدمين محليين آخرين إلى الأدلة التي سننشئها.

البارامیطر / الخيار	شنو كيعني وكيدير
`umask`	تحديد الأذونات التي تُحجب عن الملفات الجديدة.
`077`	منع صلاحيات المجموعة والآخرين افتراضياً.

النتيجة اللي كتسناها: لا يظهر نص؛ الإعداد يخص الجلسة الحالية.

تحليل النتيجة وكيفاش تقراها: هذا ليس تشفيراً ولا يغير أذونات الملفات القديمة. تقارير البروكسي وAPI قد تحتوي رموز دخول.

خطوة المثال 05: صاوب الدواسى ديال الأدلة والنوطات والتقارير

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ mkdir -p raw reports notes requests templates
```

شنو كيدير هاد الأمر: فصل الاستجابات الأصلية عن التقارير والقوالب.

البارامیطر / الخيار	شنو كيعني وكيدير
mkdir -p	إنشاء مجلدات دون مسح الموجود.
`raw`	الاستجابات الخام.
`reports`	مخرجات الأدوات.
`notes`	النطاق والتحليل.
`requests`	طلبات API وملفاتها.
`templates`	قوالب Nuclei التي راجعناها.

النتيجة اللي كتسناها: خمسة مجلدات داخل مجلد الهدف.

تحليل النتيجة وكيفاش تقراها: إعادة أمر يحفظ إلى اسم ملف موجود قد تستبدل نتيجته القديمة. استخدم مجلد جلسة جديداً في الاختبارات المتكررة.

خطوة المثال 06: حط لادريس ديال السيت فـ Variable ديال Bash

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ WEB_URL='https://www.example.com'
```

شنو كيدير هاد الأمر: تحديد أصل الويب الذي ستستعمله أمثلة الأوامر.

البارامیطر / الخيار	شنو كيعني وكيدير
`WEB_URL`	اسم متغير حساس لحالة الحروف.
`=`	إسناد دون مسافات حول علامة المساواة.
`'https://www.example.com'`	قيمة نصية؛ HTTPS هو البروتوكول www.example.com المضيف.

النتيجة اللي كتسناها: لا يظهر نص ولا يتم الاتصال.

تحليل النتيجة وكيفاش تقراها: تغيير هذا المتغير لا يغير النطاق القانوني. ولا يغير العناوين الثابتة داخل collection أو قالب أو ملف طلب.

خطوة المثال 07: تآفيشي المتغير وتتأكد منو قبل ما تصيفط أي ريكويست

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ printf 'WEB_URL=%s \
      n' "$WEB_URL"
```

شنو كيدير هاد الأمر: كشف متغير فارغ أو عنوان مكتوب خطأ قبل تشغيل الفحص.

البارامیطر / الخيار	شنو كيعني وكيدير
`printf`	طباعة بتنسيق محدد.
`%s`	مكان إدراج قيمة نصية.
`\n`	سطر جديد.
`"$WEB_URL"`	قراءة المتغير مع الحفاظ على قيمته كوسيط واحد.

النتيجة اللي كتسناها:
WEB_URL=https://www.example.com

تحليل النتيجة وكيفاش تقراها: إن كان فارغاً، أعد خطوة الإسناد في نفس الجلسة. المتغير لا ينتقل تلقائياً إلى Terminal آخر.

خطوة المثال 08: قيد الحدود والسكوب (Scope) فـ فيشي نوطات

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ nano notes/scope.md
```

شنو كيدير هاد الأمر: فتح ملف نصي لتوثيق ما يسمح به البرنامج.

الباراميطر / الخيار	شنو كيعني وكيدير
`nano`	محرر نصوص تفاعلي.
`notes/scope.md`	ملف الملاحظات الذي سنحفظه.

النتيجة اللي كتسناها: محرر؛ احفظ بـ Ctrl+O ثم Enter واخرج بـ Ctrl+X.

تحليل النتيجة وكيفاش تقراها: اكتب الأصل والمسارات المستثناة وحد الطلبات والحسابات المسموحة وهل scanning وfuzzing مسموحان. إذا كان البرنامج يطلب رأس تعريف للباحث، أضفه في كل أداة قبل استعمالها؛ الأوامر هنا لا تفترض اسمه.

خطوة المثال 09: تسجل ريسبونس وميتريكس ديال أول ريكويست بـ curl

سياق الاستخدام: طلب HTTP واحد

```
$ curl --disable --silent --show-error \
      --connect-timeout 5 --max-time 15 \
      --dump-header raw/root.headers \
      --output raw/root.html \
      --write-out 'status=%{http_code} type=%{content_type} seconds=%{time_t
  otal} ' "$WEB_URL/" > raw/root.metrics
```

شنو كيدير هاد الأمر: فصل الرؤوس والجسم وقياسات الاتصال في ثلاثة ملفات.

البارامیطر / الخيار	شنو كيعني وكيدير
`curl`	عميل HTTP؛ الطلب هنا GET.
`--disable`	تجاهل إعدادات curlrc؛ يوضع أول خيار.
--silent --show-error	إخفاء شريط التقدم مع إبقاء الأخطاء.
--connect-timeout 5	مهلة إنشاء الاتصال خمس ثوان.
--max-time 15	مهلة الطلب كله خمس عشرة ثانية.
`--dump-header`	حفظ رؤوس الاستجابة.
`--output`	حفظ جسم الصفحة.
`--write-out`	طباعة الكود والنوع والمدة بعد الطلب.
> raw/root.metrics	توجيه تلك القياسات إلى ملف؛ يستبدل محتواه إن وجد.
`\`	متابعة نفس الأمر في السطر التالي؛ لا تضف مسافة بعدها.

النتيجة اللي كتسناها: root.htmlg root.headers وroot.metricsg؛ القيمة status تختلف حسب الاستجابة.

تحليل النتيجة وكيفاش تقراها: لا يوجد -L، لذلك لا نتبع التحويل آلياً. عند 3xx نراجع Location ونطاق الوجهة. 000 أو ملف ناقص مع خطأ curl مشكلة اتصال أو TLS، وليس غياب ثغرات.

خطوة المثال 10: قرا النتيجة ديال الريكويست والميتريكس

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ cat raw/root.metrics
```

شنو كيدير هاد الأمر: عرض ملخص الطلب السابق دون اتصال جديد.

الباراميطر / الخيار	شنو كيعني وكيدير
`cat`	عرض محتوى ملف نصي.
`raw/root.metrics`	ملف القياسات الذي كتبه curl.

النتيجة اللي كتسناها: مثال توضيحي: status=200 seconds=0.250g type=text/htmlg.

تحليل النتيجة وكيفاش تقراها: الزمن يشمل الشبكة والخادم؛ بطء واحد لا يثبت ثغرة time based. HTML طبيعي للصفحة الرئيسية، لكن HTML بدل JSON في API قد يعني صفحة دخول أو تحدياً من WAF.

خطوة المثال 11: تفحص وتفلي الهيدرز (Headers) الأصليين ديال السيت

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ sed -n '1,80p' raw/root.headers
```

شنو كيدير هاد الأمر: عرض أول ثمانين سطراً من الرؤوس.

الباراميطر / الخيار	شنو كيعني وكيدير
`sed`	معالج نصوص.
`-n`	عدم طباعة كل شيء تلقائياً.
'1,80p'	اطبع السطور من 1 إلى 80.
`raw/root.headers`	مصدر البيانات المحلي.

النتيجة اللي كتسناها: سطر HTTP ورؤوس مثل content-type locationg set-cookieg.

تحليل النتيجة وكيفاش تقراها: حلل Secure HttpOnlyg SameSiteg بحسب وظيفة كل cookie. غياب رأس أمني قد يكون ملاحظة تحسين فقط؛ اشرح أثره القابل للإثبات قبل تسمية النتيجة ثغرة.

خطوة المثال 12: تصاوب ليستة ديال المسارات صغيرة ومضبوطة للفازينغ

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ printf '%s \
      n' robots.txt sitemap.xml login api/health \
      > requests/paths.txt
```

شنو كيدير هاد الأمر: تحديد أربع قيم نراجعها بدل قائمة ضخمة مجهولة.

البارامیطر / الخيار	شنو كيعني وكيدير
printf '%s\n'	طباعة كل قيمة في سطر.
robots.txt sitemap.xml login api/health	أربعة مسارات توضيحية، بلا / في البداية.
> requests/paths.txt	حفظ القائمة محلياً.

النتيجة اللي كتسناها: ملف من أربعة أسطر.

تحليل النتيجة وكيفاش تقراها: هذه المسارات لا تعني أنها موجودة على www.example.com. بدّلها بمسارات تسمح السياسة باستكشافها. robots.txt توجيه للزواحف وليس إذناً لاختبار كل ما يذكره.

خطوة المثال 13: تراني ffuf بريكويست وحدة فالثانية (بلا سبام)

سياق الاستخدام: فحص مسارات مشروط

```
$ ffuf -u 'https://www.example.com/FUZZ' \
      -w requests/paths.txt -t 1 -rate 1 \
      -timeout 10 -maxtime 60 -mc all \
      -noninteractive -of json -o reports/ffuf.json
```

شنو كيدير هاد الأمر: إرسال GET لكل قيمة في القائمة مع حفظ جميع أكواد HTTP.

البارامیطر / الخيار	شنو كيعني وكيدير
`-u`	عنوان الطلب.
`FUZZ`	مكان استبدال القيمة.
`-w`	ملف القيم.
t 1-	عامل واحد.
rate 1-	سقف طلب واحد في الثانية.
timeout 10-	مهلة كل طلب بالثواني.
maxtime 60-	حد إجمالي بالدقيقة.
mc all-	عرض كل الأكواد، حتى 404.
`-noninteractive`	تعطيل واجهة التحكم التفاعلية.
of json / -o-	صيغة التقرير ومساره.

النتيجة اللي كتسناها: نتيجة لكل مسار مكتمل؛ الكود والحجم وعدد الكلمات.

تحليل النتيجة وكيفاش تقراها: لا نضيف -r لمتابعة التحويل، ولا recursion أو auto calibration لأنها توسع الطلبات. الإعدادات الشخصية في ffufrc قد تغير السلوك؛ راجعها قبل التشغيل. أوقف الفحص عند 429 أو تكرار أخطاء الخادم.

خطوة المثال 14: قراءة نتائج ffuf حبة بحبة وفرز المسارات اللي كاينين

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ jq -r '.results[] | [.url, .status, .length, .words, .lines] | @tsv' repor
  ts/ffuf.json
```

شنو كيدير هاد الأمر: ترتيب النتائج في أعمدة نصية.

البارامیطر / الخيار	شنو كيعني وكيدير
`.results[]`	كل نتيجة محفوظة.
.status. / url	المسار وكود الرد.
length / .words / .lines.	حجم الجسم وعدد الكلمات والسطور.
r / @tsv-	نص مفصول بـtab.

النتيجة اللي كتسناها: أعمدة URL lengthg statusg linesg wordsg.

تحليل النتيجة وكيفاش تقراها: 200 بالحجم نفسه لكل المسارات قد يكون صفحة عامة تسمى soft 404. 403 ليس إثباتاً أن خلفه ملف حساس. قارن الجسم بمرجع لمسار غير موجود قبل تقرير وجود المورد.

خطوة المثال 15: تكتب قالب Nuclei محدود وموجه لهدف معين

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ cat > <<'YAML'
```

شنو كيدير هاد الأمر: فحص مؤشر كشف metadata من Git دون تنزيل المستودع.

البارامیطر / الخيار	شنو كيعني وكيدير
cat >	حفظ نص YAML كما هو.
id / info	هوية القالب ووصفه.
severity: info	تسمية ملاحظة معلوماتية؛ لا ندعي تسريب المصدر.
method: GET	قراءة المسار المحدد.
`{{BaseURL}}`	الأصل الذي تمرره للأداة.
`/.git/HEAD`	ملف metadata واحد.
redirects: false	لا تتبع التحويل.
matchers-condition: and	الشرطان معاً.
status: 200	رد ناجح.
regex / part: body	نمط يشبه مرجع فرع Git داخل الجسم.

النتيجة اللي كتسناها: ملف قالب محلي فقط.

تحليل النتيجة وكيفاش تقراها: الملف قد يحتوي metadata عامة أو تكون الاستجابة تقليداً؛ يلزم تحقق. حتى تأكيد كشف HEAD لا يثبت تسرب كامل الشيفرة. لا نتابع بتنزيل المستودع أو الأسرار ضمن هذا الإجراء.

خطوة المثال 16: تشيكي السينطاكس والصحة ديال قالب Nuclei

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ nuclei -validate -t templates/git-head.yaml -duc
```

شنو كيدير هاد الأمر: اكتشاف أخطاء YAML وبنية القالب قبل استهداف الموقع.

البارامیطر / الخيار	شنو كيعني وكيدير
`-validate`	التحقق من صحة القالب.
`-t`	اختيار ملف واحد.
`-duc`	تعطيل فحص تحديث الأداة والقوالب تلقائياً.

النتيجة اللي كتسناها: نتيجة validation أو رسالة خطأ.

تحليل النتيجة وكيفاش تقراها: نجاح الصيغة لا يعني أن القالب آمن أو أن matchers صحيحة. راجع method وأي payloadsg pathg أو OAST أو code أو headless قبل اعتماد قالب من الإنترنت.

خطوة المثال 17: تراني القالب اللي راجعتيه بمعدل ريكويست وحدة فالثانية

سياق الاستخدام: طلب فحص مشروط

```
$ nuclei -u https://www.example.com -t templates/git-head.yaml \
      -rl 1 -c 1 -bs 1 -timeout 10 -retries 0 \
      -no-interactsh -disable-redirects -duc \
      -jsonl -o reports/nuclei.jsonl
```

شنو كيدير هاد الأمر: تشغيل القالب المحدد وإخراج التطابقات إلى JSONL.

البارامیطر / الخيار	شنو كيعني وكيدير
u / -t-	هدف واحد وقالب واحد.
rl 1 / -c 1 / -bs 1-	طلب في الثانية، وقالب ومضيف واحدان بالتوازي.
timeout 10 / -retries 0-	مهلة الطلب وعدم إعادته.
`-no-interactsh`	إلغاء OAST الخاص بـInteractsh.
`-disable-redirects`	منع التحويلات في قوالب HTTP.
`-duc`	إلغاء فحص التحديث.
jsonl / -o-	كائن JSON لكل سطر في ملف النتائج.

النتيجة اللي كتسناها: تطابق إن تحققت الشروط، أو ملف فارغ عند عدم التطابق.

تحليل النتيجة وكيفاش تقراها: الملف الفارغ قد يعني عدم التطابق أو فشل اتصال؛ راجع سجل التشغيل. هذا فحص مؤشر واحد فقط. خيارات المعدل لا تحمي من قالب تخريبي؛ لا تستبدل -t بمجلد CVEs كامل ولا تفعّل code أو OAST دون مراجعة واختصاص النطاق.

خطوة المثال 18: قراءة سبب تنبيه Nuclei والماتشينغ اللي وقع

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ jq -r '["template-id", .info.severity, ."matched-at", ."matcher-name"] |
  @tsv' \
      reports/nuclei.jsonl
```

شنو كيدير هاد الأمر: عرض القالب والتصنيف ومكان التطابق.

البارامیطر / الخيار	شنو كيعني وكيدير
`."template-id"`	هوية القالب؛ الاقتباس لأن الاسم يحوي شرطة.
`.info.severity`	تصنيف القالب.
`."matched-at"`	المكان الذي تطابقت عنده الشروط.
`."matcher-name"`	اسم matcher إن كان مسمى؛ قد يكون null.
@tsv	أعمدة نصية.

النتيجة اللي كتسناها: سطر لكل تطابق؛ severity هنا info في القالب الموضح.

تحليل النتيجة وكيفاش تقراها: راجع الاستجابة الخام المحفوظة إن احتواها التقرير ثم أعد طلباً واحداً في Burp. لا تعِد نشر request أو response يحوي أسراراً؛ استخرج أقل دليل لازم.

خطوة المثال 19: توثيق النتيجة وتحديد الأثر ونسبة الثقة التقنية

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ nano notes/finding.md
```

شنو كيدير هاد الأمر: إنشاء تقرير قصير مبني على الدليل.

البارامیطر / الخيار	شنو كيعني وكيدير
`nano`	تحرير.
`notes/finding.md`	مسودة التقرير المحلية.

النتيجة اللي كتسناها: ملف تكتب فيه الهدف والطريقة والهوية والخطوات والنتيجة.

تحليل النتيجة وكيفاش تقراها: اكتب: النطاق؛ وقت الاختبار؛ الشرط المسبق؛ المتوقع؛ الفعلي؛ الدليل؛ الأثر؛ ما بقي غير مؤكد؛ معالجة مقترحة. احذف tokens وcookies والبيانات الزائدة من نسخة المشاركة. لا تسمّ البلاغ Critical اعتماداً على تسمية الأداة.

خطوة المثال 20: حساب بصمة الهاش (SHA-256) لكاع فيشيات الأدلة والتقارير

سياق الاستخدام: محلي (فالبيئة ديالك)

```
$ sha256sum raw/root.headers raw/root.html \
      > reports/evidence.sha256
```

شنو كيدير هاد الأمر: تسجيل بصمات الملفات الأساسية حتى نلاحظ أي تعديل لاحق.

البارامیطر / الخيار	شنو كيعني وكيدير
`sha256sum`	حساب SHA-256.
ملفا raw	المحتوى الذي نحسب بصمته.
> reports/evidence.sha256	حفظ البصمات محلياً.

النتيجة اللي كتسناها: سطر ببصمة لكل ملف.

تحليل النتيجة وكيفاش تقراها: البصمة تكشف تغير المحتوى إذا قارنتها بمرجع موثوق؛ لا تثبت أن المصدر صحيح أو أن المحتوى ثغرة. أضف ملفات التحقق ذات الصلة دون أرشفة أسرار التوثيق.

خلاصة تقنية (منهجية Bug Bounty 26/27 بالدارجة المغربية)

1. انضباط السكوب والبيئة ديال الخدمة:
تنظيم مساحة الخدمة فجهازك وعزل مجلدات الأدلة (raw/ baseline/ reports) والتحقق من المتغيرات قبل أي استدعاء فالتيرمينال كيحميك من أي تسريب للأسرار أو تلوث للأدلة.

2. التدقيق اليدوي كيسبق الأوتوماسيون:
قراءة استجابة HTTP وتحليل الترويسات (Headers) ورموز الحالة وشهادات TLS بيدك هي الساس باش تبني فرضيات صحيحة قبل ما تطلق أي أداة سكانير.

3. عزل حركة المرور وتوجيهها عبر البروكسي:
توجيه الريكويستات عبر وكيل Burp Suite وتجريب التغييرات حبة بحبة فـ Repeater كيخليك فاهم مزيان شنو كيوقع وكيضمن تكرار النتيجة بشكل علمي.

4. مصفوفة الصلاحيات (IDOR/BOLA):
التأكد من فصل الصلاحيات بين جوج كونطات تيست (Account A vs Account B) على مستوى منطق التطبيق (Business Logic)، حيت 200 OK بوحدها ما كتعنيش أمان أو اختراق.

5. البعد التام على الأوامر العشوائية والتخريبية:
ما تلونصيش نهائياً أوامر كتهجم بدون تحديد المعدل أو كتمسح الداتا، وحصر الفحص فقط فالطرق المنضبطة والمرخص بها ضمن سياسة البرنامج (Scope Policy).

6. التوثيق الموثوق وبصمات SHA-256:
حساب بصمة الهاش لجميع ملفات الأدلة والاستجابات باش يكون التقرير الأمني ديالك مبني على حجة قاطعة ومحمي من أي تشكيك أو تعديل.